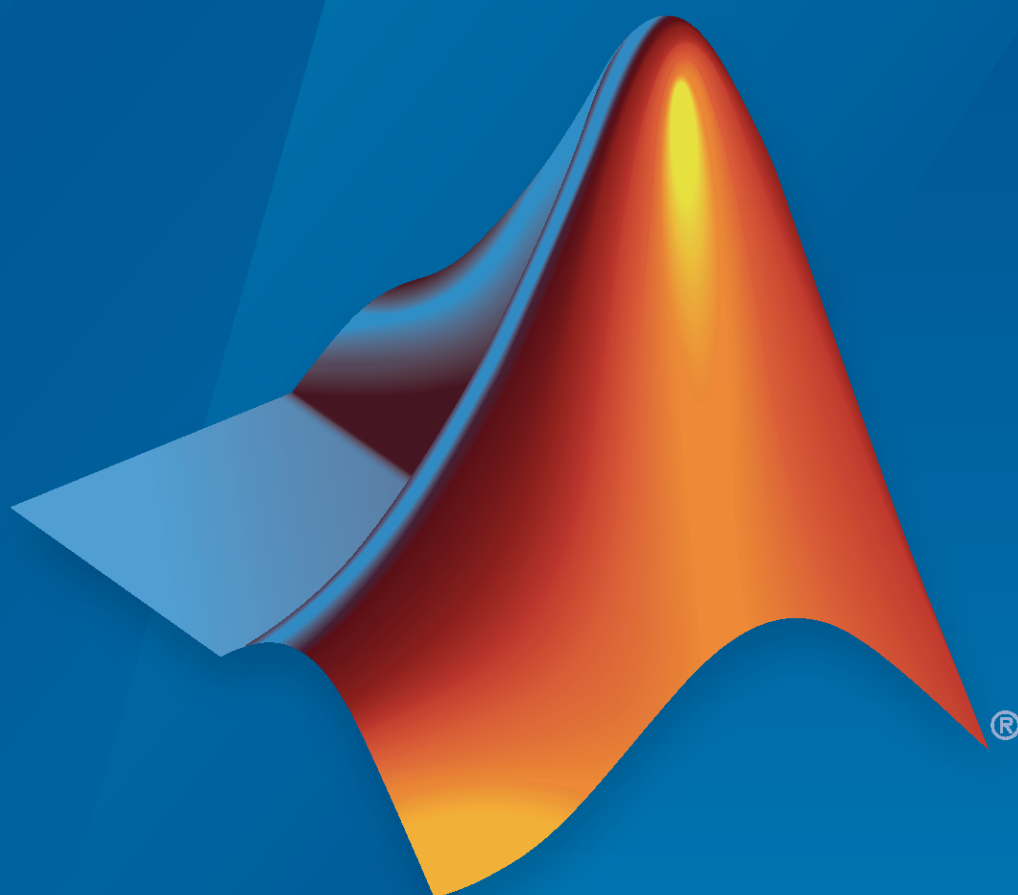


Financial Toolbox™

User's Guide



MATLAB®

R2025b



How to Contact MathWorks



Latest news: www.mathworks.com
Sales and services: www.mathworks.com/sales_and_services
User community: www.mathworks.com/matlabcentral
Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Financial Toolbox™ User's Guide

© COPYRIGHT 1995-2025 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

October 1995	First printing	
January 1998	Second printing	Revised for Version 1.1
January 1999	Third printing	Revised for Version 2.0 (Release 11)
November 2000	Fourth printing	Revised for Version 2.1.2 (Release 12)
May 2003	Online only	Revised for Version 2.3 (Release 13)
June 2004	Online only	Revised for Version 2.4 (Release 14)
August 2004	Online only	Revised for Version 2.4.1 (Release 14+)
September 2005	Fifth printing	Revised for Version 2.5 (Release 14SP3)
March 2006	Online only	Revised for Version 3.0 (Release 2006a)
September 2006	Sixth printing	Revised for Version 3.1 (Release 2006b)
March 2007	Online only	Revised for Version 3.2 (Release 2007a)
September 2007	Online only	Revised for Version 3.3 (Release 2007b)
March 2008	Online only	Revised for Version 3.4 (Release 2008a)
October 2008	Online only	Revised for Version 3.5 (Release 2008b)
March 2009	Online only	Revised for Version 3.6 (Release 2009a)
September 2009	Online only	Revised for Version 3.7 (Release 2009b)
March 2010	Online only	Revised for Version 3.7.1 (Release 2010a)
September 2010	Online only	Revised for Version 3.8 (Release 2010b)
April 2011	Online only	Revised for Version 4.0 (Release 2011a)
September 2011	Online only	Revised for Version 4.1 (Release 2011b)
March 2012	Online only	Revised for Version 4.2 (Release 2012a)
September 2012	Online only	Revised for Version 5.0 (Release 2012b)
March 2013	Online only	Revised for Version 5.1 (Release 2013a)
September 2013	Online only	Revised for Version 5.2 (Release 2013b)
March 2014	Online only	Revised for Version 5.3 (Release 2014a)
October 2014	Online only	Revised for Version 5.4 (Release 2014b)
March 2015	Online only	Revised for Version 5.5 (Release 2015a)
September 2015	Online only	Revised for Version 5.6 (Release 2015b)
March 2016	Online only	Revised for Version 5.7 (Release 2016a)
September 2016	Online only	Revised for Version 5.8 (Release 2016b)
March 2017	Online only	Revised for Version 5.9 (Release 2017a)
September 2017	Online only	Revised for Version 5.10 (Release 2017b)
March 2018	Online only	Revised for Version 5.11 (Release 2018a)
September 2018	Online only	Revised for Version 5.12 (Release 2018b)
March 2019	Online only	Revised for Version 5.13 (Release 2019a)
September 2019	Online only	Revised for Version 5.14 (Release 2019b)
March 2020	Online only	Revised for Version 5.15 (Release 2020a)
September 2020	Online only	Revised for Version 6.0 (Release 2020b)
March 2021	Online only	Revised for Version 6.1 (Release 2021a)
September 2021	Online only	Revised for Version 6.2 (Release 2021b)
March 2022	Online only	Revised for Version 6.3 (Release 2022a)
September 2022	Online only	Revised for Version 6.4 (Release 2022b)
March 2023	Online only	Revised for Version 6.5 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Rereleased for Version 25.2 (R2025b)

1

Getting Started

Financial Toolbox Product Description	1-2
Expected Users	1-3
Analyze Sets of Numbers Using Matrix Functions	1-4
Introduction	1-4
Key Definitions	1-4
Referencing Matrix Elements	1-4
Transposing Matrices	1-5
Matrix Algebra Refresher	1-7
Introduction	1-7
Adding and Subtracting Matrices	1-7
Multiplying Matrices	1-8
Dividing Matrices	1-11
Solving Simultaneous Linear Equations	1-11
Operating Element by Element	1-13
Using Input and Output Arguments with Functions	1-15
Input Arguments	1-15
Output Arguments	1-16

2

Performing Common Financial Tasks

Handle and Convert Dates	2-2
Date Formats	2-2
Date Conversions	2-3
Current Date and Time	2-7
Determining Specific Dates	2-8
Determining Holidays	2-8
Determining Cash-Flow Dates	2-9
Analyzing and Computing Cash Flows	2-11
Introduction	2-11
Interest Rates/Rates of Return	2-11
Present or Future Values	2-12
Depreciation	2-13
Annuities	2-13

Pricing and Computing Yields for Fixed-Income Securities	2-15
Introduction	2-15
Fixed-Income Terminology	2-15
Framework	2-18
Default Parameter Values	2-18
Coupon Date Calculations	2-20
Yield Conventions	2-21
Pricing Functions	2-21
Yield Functions	2-21
Fixed-Income Sensitivities	2-22
Treasury Bills Defined	2-25
Computing Treasury Bill Price and Yield	2-26
Introduction	2-26
Treasury Bill Repurchase Agreements	2-26
Treasury Bill Yields	2-27
Term Structure of Interest Rates	2-29
Returns with Negative Prices	2-32
Negative Price Conversion	2-32
Analysis of Negative Price Returns	2-33
Visualization of Complex Returns	2-35
Conclusion	2-38
Pricing and Analyzing Equity Derivatives	2-39
Introduction	2-39
Sensitivity Measures	2-39
Analysis Models	2-40
About Life Tables	2-44
Life Tables Theory	2-44
Case Study for Life Tables Analysis	2-46
Machine Learning for Statistical Arbitrage: Introduction	2-48
Machine Learning for Statistical Arbitrage I: Data Management and Visualization	2-50
Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development	2-59
Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction	2-69
Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data	2-78

Analyzing Portfolios	3-2
Portfolio Optimization Functions	3-3
Portfolio Construction Examples	3-5
Introduction	3-5
Efficient Frontier Example	3-5
Portfolio Selection and Risk Aversion	3-7
Introduction	3-7
Optimal Risky Portfolio	3-8
portopt Migration to Portfolio Object	3-11
Migrate portopt Without Output Arguments	3-11
Migrate portopt with Output Arguments	3-12
Migrate portopt for Target Returns Within Range of Efficient Portfolio Returns	3-13
Migrate portopt for Target Return Outside Range of Efficient Portfolio Returns	3-14
Migrate portopt Using portcons Output for ConSet	3-15
Integrate Output from portcons, pcalims, pcglims, and pcgcomp with a Portfolio Object	3-17
Constraint Specification Using a Portfolio Object	3-19
Constraints for Efficient Frontier	3-19
Linear Constraint Equations	3-21
Specifying Group Constraints	3-24
Active Returns and Tracking Error Efficient Frontier	3-27

Mean-Variance Portfolio Optimization Tools

Portfolio Optimization Theory	4-4
Portfolio Optimization Problems	4-4
Portfolio Problem Specification	4-4
Return Proxy	4-5
Risk Proxy	4-6
Supported Constraints for Portfolio Optimization Using Portfolio Objects	4-9
Linear Inequality Constraints	4-9
Linear Equality Constraints	4-10
'Simple' Bound Constraints	4-10
'Conditional' Bound Constraints	4-11
Budget Constraints	4-11
Conditional Budget Constraints	4-12
Group Constraints	4-12

Group Ratio Constraints	4-13
Average Turnover Constraints	4-14
One-Way Turnover Constraints	4-14
Tracking Error Constraints	4-15
Cardinality Constraints	4-16
Default Portfolio Problem	4-17
Portfolio Object Workflow	4-18
Portfolio Object	4-20
Portfolio Object Properties and Functions	4-20
Working with Portfolio Objects	4-20
Setting and Getting Properties	4-20
Displaying Portfolio Objects	4-21
Saving and Loading Portfolio Objects	4-21
Estimating Efficient Portfolios and Frontiers	4-21
Arrays of Portfolio Objects	4-22
Subclassing Portfolio Objects	4-23
Conventions for Representation of Data	4-23
Creating the Portfolio Object	4-25
Syntax	4-25
Portfolio Problem Sufficiency	4-25
Portfolio Function Examples	4-26
Common Operations on the Portfolio Object	4-33
Naming a Portfolio Object	4-33
Configuring the Assets in the Asset Universe	4-33
Setting Up a List of Asset Identifiers	4-33
Truncating and Padding Asset Lists	4-35
Setting Up an Initial or Current Portfolio	4-37
Setting Up a Tracking Portfolio	4-40
Asset Returns and Moments of Asset Returns Using Portfolio Object	4-42
Assignment Using the Portfolio Function	4-42
Assignment Using the setAssetMoments Function	4-43
Scalar Expansion of Arguments	4-44
Estimating Asset Moments from Prices or Returns	4-45
Estimating Asset Moments with Missing Data	4-47
Estimating Asset Moments from Time Series Data	4-49
Working with a Riskless Asset	4-52
Working with Transaction Costs	4-54
Setting Transaction Costs Using the Portfolio Function	4-54
Setting Transaction Costs Using the setCosts Function	4-54
Setting Transaction Costs with Scalar Expansion	4-56
Working with Portfolio Constraints Using Defaults	4-58
Setting Default Constraints for Portfolio Weights Using Portfolio Object	4-58

Working with 'Simple' Bound Constraints Using Portfolio Object	4-62
Setting 'Simple' Bounds Using the Portfolio Function	4-62
Setting 'Simple' Bounds Using the setBounds Function	4-62
Setting 'Simple' Bounds Using the Portfolio Function or setBounds Function	4-63
Working with Budget Constraints Using Portfolio Object	4-65
Setting Budget Constraints Using the Portfolio Function	4-65
Setting Budget Constraints Using the setBudget Function	4-65
Working with Conditional Budget Constraints Using Portfolio Object . .	4-67
Setting Conditional Budget Constraints Using the Portfolio Function . . .	4-67
Setting Conditional Budget Constraints Using the setConditionalBudget Function	4-67
Working with Group Constraints Using Portfolio Object	4-69
Setting Group Constraints Using the Portfolio Function	4-69
Setting Group Constraints Using the setGroups and addGroups Functions	4-69
Working with Group Ratio Constraints Using Portfolio Object	4-72
Setting Group Ratio Constraints Using the Portfolio Function	4-72
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	4-73
Working with Linear Equality Constraints Using Portfolio Object	4-75
Setting Linear Equality Constraints Using the Portfolio Function	4-75
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	4-75
Working with Linear Inequality Constraints Using Portfolio Object	4-78
Setting Linear Inequality Constraints Using the Portfolio Function	4-78
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	4-78
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects	4-81
Setting 'Conditional' BoundType Constraints Using the setBounds Function	4-81
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	4-82
Working with Average Turnover Constraints Using Portfolio Object . . .	4-85
Setting Average Turnover Constraints Using the Portfolio Function	4-85
Setting Average Turnover Constraints Using the setTurnover Function . .	4-85
Working with One-Way Turnover Constraints Using Portfolio Object . . .	4-88
Setting One-Way Turnover Constraints Using the Portfolio Function	4-88
Setting Turnover Constraints Using the setOneWayTurnover Function . .	4-88
Working with Tracking Error Constraints Using Portfolio Object	4-91
Setting Tracking Error Constraints Using the Portfolio Function	4-91
Setting Tracking Error Constraints Using the setTrackingError Function	4-91

Validate the Portfolio Problem for Portfolio Object	4-94
Validating a Portfolio Set	4-94
Validating Portfolios	4-95
Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object	4-98
Obtaining Portfolios Along the Entire Efficient Frontier	4-99
Obtaining Endpoints of the Efficient Frontier	4-102
Obtaining Efficient Portfolios for Target Returns	4-105
Obtaining Efficient Portfolios for Target Risks	4-108
Efficient Portfolio That Maximizes Sharpe Ratio	4-111
Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization	4-114
Using 'lcprog' and 'quadprog'	4-114
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	4-115
Solver Guidelines for Portfolio Objects	4-115
Solver Guidelines for Custom Objective Problems Using Portfolio Objects	4-119
Estimate Efficient Frontiers for Portfolio Object	4-122
Obtaining Portfolio Risks and Returns	4-122
Plotting the Efficient Frontier for a Portfolio Object	4-125
Postprocessing Results to Set Up Tradable Portfolios	4-130
When to Use Portfolio Objects Over Optimization Toolbox	4-132
Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-134
Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-135
Use Optimization Toolbox	4-136
Comparison of Methods for Covariance Estimation	4-138
Choose MINLP Solvers for Portfolio Problems	4-140
Troubleshooting Portfolio Optimization Results	4-145
Portfolio Object Destroyed When Modifying	4-145
Optimization Fails with "Bad Pivot" Message	4-145
Speed of Optimization	4-145
Matrix Incompatibility and "Non-Conformable" Errors	4-145
Missing Data Estimation Fails	4-145
mv_optim_transform Errors	4-145
solveContinuousCustomObjProb or solveMICustomObjProb Errors	4-146
Efficient Portfolios Do Not Make Sense	4-146
Efficient Frontiers Do Not Make Sense	4-146
Troubleshooting estimateCustomObjectivePortfolio	4-148
Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints	4-148

Role of Convexity in Portfolio Problems	4-157
Examples of Convex Functions	4-158
Examples of Concave Functions	4-159
Examples of Nonconvex Functions	4-159
Portfolio Optimization Examples Using Financial Toolbox	4-161
Asset Allocation Case Study	4-180
Portfolio Optimization with Semicontinuous and Cardinality Constraints	4-190
Portfolio Optimization Against a Benchmark	4-202
Portfolio Analysis with Turnover Constraints	4-211
Leverage in Portfolio Optimization with a Risk-Free Asset	4-217
Black-Litterman Portfolio Optimization Using Financial Toolbox	4-222
Portfolio Optimization Using Factor Models	4-231
Backtest Investment Strategies Using Financial Toolbox	4-238
Backtest Investment Strategies with Trading Signals	4-251
Portfolio Optimization Using Social Performance Measure	4-264
Diversify ESG Portfolios	4-271
Risk Budgeting Portfolio	4-286
Backtest Using Risk-Based Equity Indexation	4-291
Create Hierarchical Risk Parity Portfolio	4-296
Backtest Strategies Using Deep Learning	4-302
Backtest with Brinson Attribution to Evaluate Portfolio Performance	4-315
Analyze Performance Attribution Using Brinson Model	4-323
Diversify Portfolios Using Custom Objective	4-331
Solve Tracking Error Portfolio Problems	4-343
Solve Problem for Minimum Tracking Error with Net Return Constraint	4-349
Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty	4-351
Risk Parity or Budgeting with Constraints	4-357

Single Period Goal-Based Wealth Management	4-362
Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods	4-367
Multiperiod Goal-Based Wealth Management Using Reinforcement Learning	4-379
Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting	4-394
Mixed-Integer Mean-Variance Portfolio Optimization Problem	4-402
Deep Reinforcement Learning for Optimal Trade Execution	4-407
Backtest Investment Strategies Using datetime and calendarDuration	4-451
Adding Constraints to Satisfy UCITS Directive	4-457

CVaR Portfolio Optimization Tools

5

Portfolio Optimization Theory	5-3
Portfolio Optimization Problems	5-3
Portfolio Problem Specification	5-3
Return Proxy	5-4
Risk Proxy	5-5
Supported Constraints for Portfolio Optimization Using PortfolioCVaR	
Object	5-8
Linear Inequality Constraints	5-8
Linear Equality Constraints	5-9
'Simple' Bound Constraints	5-9
'Conditional' Bound Constraints	5-10
Budget Constraints	5-10
Conditional Budget Constraints	5-11
Group Constraints	5-11
Group Ratio Constraints	5-12
Average Turnover Constraints	5-13
One-way Turnover Constraints	5-13
Cardinality Constraints	5-14
Default Portfolio Problem	5-15
PortfolioCVaR Object Workflow	5-16
PortfolioCVaR Object	5-17
PortfolioCVaR Object Properties and Functions	5-17
Working with PortfolioCVaR Objects	5-17
Setting and Getting Properties	5-18
Displaying PortfolioCVaR Objects	5-18

Saving and Loading PortfolioCVaR Objects	5-18
Estimating Efficient Portfolios and Frontiers	5-18
Arrays of PortfolioCVaR Objects	5-19
Subclassing PortfolioCVaR Objects	5-20
Conventions for Representation of Data	5-20
Creating the PortfolioCVaR Object	5-22
Syntax	5-22
PortfolioCVaR Problem Sufficiency	5-22
PortfolioCVaR Function Examples	5-23
Common Operations on the PortfolioCVaR Object	5-29
Naming a PortfolioCVaR Object	5-29
Configuring the Assets in the Asset Universe	5-29
Setting Up a List of Asset Identifiers	5-29
Truncating and Padding Asset Lists	5-31
Setting Up an Initial or Current Portfolio	5-33
Asset Returns and Scenarios Using PortfolioCVaR Object	5-36
How Stochastic Optimization Works	5-36
What Are Scenarios?	5-36
Setting Scenarios Using the PortfolioCVaR Function	5-37
Setting Scenarios Using the setScenarios Function	5-38
Estimating the Mean and Covariance of Scenarios	5-38
Simulating Normal Scenarios	5-39
Simulating Normal Scenarios from Returns or Prices	5-39
Simulating Normal Scenarios with Missing Data	5-40
Simulating Normal Scenarios from Time Series Data	5-42
Simulating Normal Scenarios with Mean and Covariance	5-44
Working with a Riskless Asset	5-46
Working with Transaction Costs	5-47
Setting Transaction Costs Using the PortfolioCVaR Function	5-47
Setting Transaction Costs Using the setCosts Function	5-47
Setting Transaction Costs with Scalar Expansion	5-49
Working with CVaR Portfolio Constraints Using Defaults	5-51
Setting Default Constraints for Portfolio Weights Using PortfolioCVaR Object	5-51
Working with 'Simple' Bound Constraints Using PortfolioCVaR Object	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function	5-55
Setting 'Simple' Bounds Using the setBounds Function	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function or setBounds Function	5-56
Working with Budget Constraints Using PortfolioCVaR Object	5-58
Setting Budget Constraints Using the PortfolioCVaR Function	5-58
Setting Budget Constraints Using the setBudget Function	5-58

Working with Conditional Budget Constraints Using PortfolioCvAR Object	
Setting Conditional Budget Constraints Using the PortfolioCvAR Function	5-60
Setting Conditional Budget Constraints Using the setConditionalBudget Function	5-60
Working with Group Constraints Using PortfolioCvAR Object	5-62
Setting Group Constraints Using the PortfolioCvAR Function	5-62
Setting Group Constraints Using the setGroups and addGroups Functions	5-62
Working with Group Ratio Constraints Using PortfolioCvAR Object	5-65
Setting Group Ratio Constraints Using the PortfolioCvAR Function	5-65
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	5-66
Working with Linear Equality Constraints Using PortfolioCvAR Object	5-68
Setting Linear Equality Constraints Using the PortfolioCvAR Function	5-68
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	5-68
Working with Linear Inequality Constraints Using PortfolioCvAR Object	5-70
Setting Linear Inequality Constraints Using the PortfolioCvAR Function	5-70
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	5-70
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioCvAR Objects	5-72
Setting 'Conditional' BoundType Constraints Using the setBounds Function	5-72
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	5-73
Working with Average Turnover Constraints Using PortfolioCvAR Object	5-75
Setting Average Turnover Constraints Using the PortfolioCvAR Function	5-75
Setting Average Turnover Constraints Using the setTurnover Function	5-75
Working with One-Way Turnover Constraints Using PortfolioCvAR Object	5-78
Setting One-Way Turnover Constraints Using the PortfolioCvAR Function	5-78
Setting Turnover Constraints Using the setOneWayTurnover Function	5-78
Validate the CVaR Portfolio Problem	5-81
Validating a CVaR Portfolio Set	5-81
Validating CVaR Portfolios	5-82
Estimate Efficient Portfolios for Entire Frontier for PortfolioCvAR Object	5-85

Obtaining Portfolios Along the Entire Efficient Frontier	5-86
Obtaining Endpoints of the Efficient Frontier	5-89
Obtaining Efficient Portfolios for Target Returns	5-92
Obtaining Efficient Portfolios for Target Risks	5-95
Choosing and Controlling the Solver for PortfolioCVaR Optimizations .	5-98
Using 'TrustRegionCP', 'ExtendedCP', and 'cuttingplane' SolverTypes . . .	5-98
Using 'fmincon' SolverType	5-99
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	5-100
Solver Guidelines for PortfolioCVaR Objects	5-100
Estimate Efficient Frontiers for PortfolioCVaR Object	5-105
Obtaining CVaR Portfolio Risks and Returns	5-105
Obtaining Portfolio Standard Deviation and VaR	5-106
Plotting the Efficient Frontier for a PortfolioCVaR Object	5-109
Postprocessing Results to Set Up Tradable Portfolios	5-115
Working with Other Portfolio Objects	5-118
Troubleshooting CVaR Portfolio Optimization Results	5-121
PortfolioCVaR Object Destroyed When Modifying	5-121
Matrix Incompatibility and "Non-Conformable" Errors	5-121
CVaR Portfolio Optimization Warns About "Max Iterations"	5-121
CVaR Portfolio Optimization Errors with "Could Not Solve" Message . .	5-122
Missing Data Estimation Fails	5-122
cvar_optim_transform Errors	5-122
Efficient Portfolios Do Not Make Sense	5-123
Hedging Using CVaR Portfolio Optimization	5-125
Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio	5-137
Mixed-Integer CVaR Portfolio Optimization Problem	5-141
Bond Portfolio CVaR Optimization Using Diebold-Li Model	5-146

MAD Portfolio Optimization Tools

6

Portfolio Optimization Theory	6-3
Portfolio Optimization Problems	6-3
Portfolio Problem Specification	6-3
Return Proxy	6-4
Risk Proxy	6-5

Supported Constraints for Portfolio Optimization Using PortfolioMAD	
Object	6-8
Linear Inequality Constraints	6-8
Linear Equality Constraints	6-9
'Simple' Bound Constraints	6-9
'Conditional' Bound Constraints	6-10
Budget Constraints	6-10
Conditional Budget Constraints	6-11
Group Constraints	6-11
Group Ratio Constraints	6-12
Average Turnover Constraints	6-13
One-way Turnover Constraints	6-13
Cardinality Constraints	6-14
Default Portfolio Problem	6-15
PortfolioMAD Object Workflow	6-16
PortfolioMAD Object	6-17
PortfolioMAD Object Properties and Functions	6-17
Working with PortfolioMAD Objects	6-17
Setting and Getting Properties	6-18
Displaying PortfolioMAD Objects	6-18
Saving and Loading PortfolioMAD Objects	6-18
Estimating Efficient Portfolios and Frontiers	6-18
Arrays of PortfolioMAD Objects	6-19
Subclassing PortfolioMAD Objects	6-20
Conventions for Representation of Data	6-20
Creating the PortfolioMAD Object	6-22
Syntax	6-22
PortfolioMAD Problem Sufficiency	6-22
PortfolioMAD Function Examples	6-23
Common Operations on the PortfolioMAD Object	6-29
Naming a PortfolioMAD Object	6-29
Configuring the Assets in the Asset Universe	6-29
Setting Up a List of Asset Identifiers	6-29
Truncating and Padding Asset Lists	6-31
Setting Up an Initial or Current Portfolio	6-33
Asset Returns and Scenarios Using PortfolioMAD Object	6-35
How Stochastic Optimization Works	6-35
What Are Scenarios?	6-35
Setting Scenarios Using the PortfolioMAD Function	6-36
Setting Scenarios Using the setScenarios Function	6-37
Estimating the Mean and Covariance of Scenarios	6-37
Simulating Normal Scenarios	6-38
Simulating Normal Scenarios from Returns or Prices	6-38
Simulating Normal Scenarios with Missing Data	6-39
Simulating Normal Scenarios from Time Series Data	6-41
Simulating Normal Scenarios with Mean and Covariance	6-43
Working with a Riskless Asset	6-45

Working with Transaction Costs	6-46
Setting Transaction Costs Using the PortfolioMAD Function	6-46
Setting Transaction Costs Using the setCosts Function	6-46
Setting Transaction Costs with Scalar Expansion	6-48
Working with MAD Portfolio Constraints Using Defaults	6-50
Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object	6-50
Working with 'Simple' Bound Constraints Using PortfolioMAD Object .	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function	6-54
Setting 'Simple' Bounds Using the setBounds Function	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function or setBounds Function	6-55
Working with Budget Constraints Using PortfolioMAD Object	6-57
Setting Budget Constraints Using the PortfolioMAD Function	6-57
Setting Budget Constraints Using the setBudget Function	6-57
Working with Conditional Budget Constraints Using PortfolioMAD Object	6-59
Setting Conditional Budget Constraints Using the PortfolioMAD Function	6-59
Setting Conditional Budget Constraints Using the setConditionalBudget Function	6-59
Working with Group Constraints Using PortfolioMAD Object	6-61
Setting Group Constraints Using the PortfolioMAD Function	6-61
Setting Group Constraints Using the setGroups and addGroups Functions	6-61
Working with Group Ratio Constraints Using PortfolioMAD Object	6-64
Setting Group Ratio Constraints Using the PortfolioMAD Function	6-64
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	6-65
Working with Linear Equality Constraints Using PortfolioMAD Object	6-67
Setting Linear Equality Constraints Using the PortfolioMAD Function ..	6-67
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	6-67
Working with Linear Inequality Constraints Using PortfolioMAD Object	6-69
Setting Linear Inequality Constraints Using the PortfolioMAD Function .	6-69
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	6-69
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects	6-71
Setting 'Conditional' BoundType Constraints Using the setBounds Function	6-71
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	6-72

Working with Average Turnover Constraints Using PortfolioMAD Object	6-74
Setting Average Turnover Constraints Using the PortfolioMAD Function	6-74
Setting Average Turnover Constraints Using the setTurnover Function	6-74
Working with One-Way Turnover Constraints Using PortfolioMAD Object	6-77
Setting One-Way Turnover Constraints Using the PortfolioMAD Function	6-77
Setting Turnover Constraints Using the setOneWayTurnover Function	6-77
Validate the MAD Portfolio Problem	6-80
Validating a MAD Portfolio Set	6-80
Validating MAD Portfolios	6-81
Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object	6-84
Obtaining Portfolios Along the Entire Efficient Frontier	6-85
Obtaining Endpoints of the Efficient Frontier	6-87
Mixed-Integer MAD Portfolio Optimization Problem	6-89
Obtaining Efficient Portfolios for Target Returns	6-93
Obtaining Efficient Portfolios for Target Risks	6-96
Choosing and Controlling the Solver for PortfolioMAD Optimizations	6-99
Using 'TrustRegionCP' and 'ExtendedCP' SolverTypes	6-99
Using 'fmincon' SolverType	6-100
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	6-101
Solver Guidelines for PortfolioMAD Objects	6-101
Estimate Efficient Frontiers for PortfolioMAD Object	6-105
Obtaining MAD Portfolio Risks and Returns	6-105
Obtaining the PortfolioMAD Standard Deviation	6-106
Plotting the Efficient Frontier for a PortfolioMAD Object	6-108
Postprocessing Results to Set Up Tradable Portfolios	6-113
Working with Other Portfolio Objects	6-115
Troubleshooting MAD Portfolio Optimization Results	6-118
PortfolioMAD Object Destroyed When Modifying	6-118
Matrix Incompatibility and "Non-Conformable" Errors	6-118
Missing Data Estimation Fails	6-118
mad_optim_transform Errors	6-118
Efficient Portfolios Do Not Make Sense	6-119

Performance Metrics Overview	7-2
Performance Metrics Types	7-2
Performance Metrics Illustration	7-3
Using the Sharpe Ratio	7-5
Using the Information Ratio	7-7
Using Tracking Error	7-8
Using Risk-Adjusted Return	7-9
Using Sample and Expected Lower Partial Moments	7-11
Introduction	7-11
Sample Lower Partial Moments	7-11
Expected Lower Partial Moments	7-12
Using Maximum and Expected Maximum Drawdown	7-14
Introduction	7-14
Maximum Drawdown	7-14
Expected Maximum Drawdown	7-16

Estimation of Transition Probabilities	8-2
Introduction	8-2
Estimate Transition Probabilities	8-2
Estimate Transition Probabilities for Different Rating Scales	8-4
Working with a Transition Matrix Containing NR Rating	8-6
Estimate Point-in-Time and Through-the-Cycle Probabilities	8-10
Estimate t-Year Default Probabilities	8-12
Estimate Bootstrap Confidence Intervals	8-13
Group Credit Ratings	8-15
Work with Nonsquare Matrices	8-17
Remove Outliers	8-18
Estimate Probabilities for Different Segments	8-19
Work with Large Datasets	8-20
Forecasting Corporate Default Rates	8-23
Credit Quality Thresholds	8-45
Introduction	8-45
Compute Credit Quality Thresholds	8-45
Visualize Credit Quality Thresholds	8-46

About Credit Scorecards	8-49
What Is a Credit Scorecard?	8-49
Credit Scorecard Development Process	8-50
Credit Scorecard Modeling Workflow	8-52
Credit Scorecard Modeling Using Observation Weights	8-55
Credit Scorecard Modeling with Missing Values	8-57
Troubleshooting Credit Scorecard Results	8-64
Predictor Name Is Unspecified and the Parser Returns an Error	8-64
Using bininfo or plotbins Before Binning	8-64
If Categorical Data Is Given as Numeric	8-66
NaNs Returned When Scoring a “Test” Dataset	8-68
Case Study for Credit Scorecard Analysis	8-71
Credit Scorecards with Constrained Logistic Regression Coefficients ..	8-91
Credit Default Swap (CDS)	8-100
Bootstrapping a Default Probability Curve	8-101
Finding Breakeven Spread for New CDS Contract	8-104
Valuing an Existing CDS Contract	8-107
Converting from Running to Upfront	8-109
Bootstrapping from Inverted Market Curves	8-111
Visualize Transitions Data for transprob	8-114
Impute Missing Data in the Credit Scorecard Workflow Using the k-Nearest Neighbors Algorithm	8-121
Impute Missing Data in the Credit Scorecard Workflow Using the Random Forest Algorithm	8-127
Treat Missing Data in a Credit Scorecard Workflow Using MATLAB fillmissing	8-132

Regression with Missing Data

9

Multivariate Normal Regression	9-2
Introduction	9-2
Multivariate Normal Linear Regression	9-2
Maximum Likelihood Estimation	9-3
Special Case of Multiple Linear Regression Model	9-4

Least-Squares Regression	9-4
Mean and Covariance Estimation	9-4
Convergence	9-4
Fisher Information	9-4
Statistical Tests	9-5
Maximum Likelihood Estimation with Missing Data	9-7
Introduction	9-7
ECM Algorithm	9-7
Standard Errors	9-8
Data Augmentation	9-8
Multivariate Normal Regression Functions	9-10
Multivariate Normal Regression Without Missing Data	9-11
Multivariate Normal Regression With Missing Data	9-11
Least-Squares Regression With Missing Data	9-11
Multivariate Normal Parameter Estimation With Missing Data	9-12
Support Functions	9-12
Multivariate Normal Regression Types	9-13
Regressions	9-13
Multivariate Normal Regression	9-13
Multivariate Normal Regression Without Missing Data	9-13
Multivariate Normal Regression With Missing Data	9-14
Least-Squares Regression	9-14
Least-Squares Regression Without Missing Data	9-14
Least-Squares Regression With Missing Data	9-14
Covariance-Weighted Least Squares	9-14
Covariance-Weighted Least Squares Without Missing Data	9-15
Covariance-Weighted Least Squares With Missing Data	9-15
Feasible Generalized Least Squares	9-15
Feasible Generalized Least Squares Without Missing Data	9-15
Feasible Generalized Least Squares With Missing Data	9-16
Seemingly Unrelated Regression	9-16
Seemingly Unrelated Regression Without Missing Data	9-17
Seemingly Unrelated Regression With Missing Data	9-17
Mean and Covariance Parameter Estimation	9-17
Troubleshooting Multivariate Normal Regression	9-18
Biased Estimates	9-18
Requirements	9-18
Slow Convergence	9-18
Nonrandom Residuals	9-19
Nonconvergence	9-19
Portfolios with Missing Data	9-21
Valuation with Missing Data	9-26
Introduction	9-26
Capital Asset Pricing Model	9-26
Estimation of the CAPM	9-27
Estimation with Missing Data	9-27
Estimation of Some Technology Stock Betas	9-27
Grouped Estimation of Some Technology Stock Betas	9-30
References	9-32

Capital Asset Pricing Model with Missing Data	9-34
---	------

Solving Sample Problems

10

Sensitivity of Bond Prices to Interest Rates	10-2
Bond Portfolio for Hedging Duration and Convexity	10-6
Bond Prices and Yield Curve Parallel Shifts	10-9
Bond Prices and Yield Curve Nonparallel Shifts	10-12
Greek-Neutral Portfolios of European Stock Options	10-14
Term Structure Analysis and Interest-Rate Swaps	10-18
Plotting an Efficient Frontier Using portopt	10-22
Plotting Sensitivities of an Option	10-25
Plotting Sensitivities of a Portfolio of Options	10-27
Bond Portfolio Optimization Using Portfolio Object	10-30
Hedge Options Using Reinforcement Learning Toolbox	10-40
Hedge Using Monte Carlo Simulation	10-49
Using Extreme Value Theory and Copula Fitting to Generate Synthetic Data	10-61

Using Financial Timetables

11

Convert Financial Time Series Objects (fints) to Timetables	11-2
Create Time Series	11-2
Index an Object	11-3
Transform Time Series	11-3
Convert Time Series	11-4
Merge Time Series	11-5
Analyze Time Series	11-5
Data Extraction	11-6
Use Timetables in Finance	11-7

12

Trading Calendars User Interface	12-2
UICalendar User Interface	12-4
Using UICalendar in Standalone Mode	12-4
Using UICalendar with an Application	12-4

Technical Analysis**13**

Technical Indicators	13-2
-----------------------------------	-------------

Stochastic Differential Equations**14**

SDEs	14-2
SDE Modeling	14-2
Trials vs. Paths	14-3
NTrials, NPeriods, and NSteps	14-3
SDE Class Hierarchy	14-5
SDE Models	14-7
Introduction	14-7
Creating SDE Objects	14-7
Drift and Diffusion	14-10
Available SDE Models	14-11
SDE Simulation and Interpolation Methods	14-13
Base SDE Models	14-16
Overview	14-16
Specify Base Stochastic Differential Equation (SDE) Model	14-16
Drift and Diffusion Models	14-19
Overview	14-19
Specify Drift and Diffusion Rate Functions	14-19
Specify SDEDDO with Drift and Diffusion Functions	14-20
Linear Drift Models	14-22
Overview	14-22
Specify SDELD Model	14-22
Parametric Models	14-24
Creating Brownian Motion (BM) Models	14-24
Specify Brownian Motion Model	14-24
Creating Constant Elasticity of Variance (CEV) Models	14-25

Creating Geometric Brownian Motion (GBM) Models	14-25
Creating Stochastic Differential Equations from Mean-Reverting Drift (SDEMRD) Models	14-26
Creating Cox-Ingersoll-Ross (CIR) Square Root Diffusion Models	14-27
Creating Hull-White/Vasicek (HWV) Gaussian Diffusion Models	14-28
Creating Heston Stochastic Volatility Models	14-29
Simulating Equity Prices	14-31
Simulating Multidimensional Market Models	14-31
Induce Dependence and Correlation Between States	14-41
Dynamic Behavior of Market Parameters	14-43
Price European Stock Options Using Monte Carlo Simulation	14-47
Simulating Interest Rates	14-50
Simulate Interest Rates Through Interpolation	14-50
Simulate Positive Interest Rates	14-54
Stratified Sampling	14-58
Quasi-Monte Carlo Simulation	14-63
Performance Considerations	14-65
Managing Memory	14-65
Enhancing Performance	14-66
Optimizing Accuracy: About Solution Precision and Error	14-66
Price American Basket Options Using Standard Monte Carlo and Quasi- Monte Carlo Simulation	14-71
Volatility Modeling for Soft Commodities	14-88
Improving Performance of Monte Carlo Simulation with Parallel Computing	14-111

Functions

15

Bibliography

A

Bibliography	A-2
Bond Pricing and Yields	A-2
Term Structure of Interest Rates	A-2
Derivatives Pricing and Yields	A-3
Portfolio Analysis	A-3
Investment Performance Metrics	A-3
Financial Statistics	A-4
Standard References	A-4
Credit Risk Analysis	A-5

Credit Derivatives	A-5
Portfolio Optimization	A-5
Stochastic Differential Equations	A-6
Life Tables	A-6

Getting Started

- “Financial Toolbox Product Description” on page 1-2
- “Expected Users” on page 1-3
- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Financial Toolbox Product Description

Analyze financial data and develop financial models

Financial Toolbox provides functions for the mathematical modeling and statistical analysis of financial data. You can analyze, backtest, and optimize investment portfolios taking into account turnover, transaction costs, semi-continuous constraints, and minimum or maximum number of assets. The toolbox enables you to estimate risk, model credit scorecards, analyze yield curves, price fixed-income instruments and European options, and measure investment performance.

Stochastic differential equation (SDE) tools let you model and simulate a variety of stochastic processes. Time series analysis functions let you perform transformations or regressions with missing data and convert between different trading calendars and day-count conventions.

Expected Users

In general, this guide assumes experience working with financial derivatives and some familiarity with the underlying models.

In designing Financial Toolbox documentation, we assume that your title is like one of these:

- Analyst, quantitative analyst
- Risk manager
- Portfolio manager
- Asset allocator
- Financial engineer
- Trader
- Student, professor, or other academic

We also assume that your background, education, training, and responsibilities match some aspects of this profile:

- Finance, economics, perhaps accounting
- Engineering, mathematics, physics, other quantitative sciences
- Focus on quantitative approaches to financial problems

Analyze Sets of Numbers Using Matrix Functions

In this section...

"Introduction" on page 1-4

"Key Definitions" on page 1-4

"Referencing Matrix Elements" on page 1-4

"Transposing Matrices" on page 1-5

Introduction

Many financial analysis procedures involve *sets* of numbers; for example, a portfolio of securities at various prices and yields. Matrices, matrix functions, and matrix algebra are the most efficient ways to analyze sets of numbers and their relationships. Spreadsheets focus on individual cells and the relationships between cells. While you can think of a set of spreadsheet cells (a range of rows and columns) as a matrix, a matrix-oriented tool like MATLAB® software manipulates sets of numbers more quickly, easily, and naturally. For more information, see "Matrix Algebra Refresher" on page 1-7.

Key Definitions

Matrix

A rectangular array of numeric or algebraic quantities subject to mathematical operations; the regular formation of elements into rows and columns. Described as a "*m*-by-*n*" matrix, with *m* the number of rows and *n* the number of columns. The description is always "row-by-column." For example, here is a 2-by-3 matrix of two bonds (the rows) with different par values, coupon rates, and coupon payment frequencies per year (the columns) entered using MATLAB notation:

```
Bonds = [1000    0.06    2
          500     0.055   4]
```

Vector

A matrix with only one row or column. Described as a "1-by-*n*" or "*m*-by-1" matrix. The description is always "row-by-column." For example, here is a 1-by-4 vector of cash flows in MATLAB notation:

```
Cash = [1500    4470    5280   -1299]
```

Scalar

A 1-by-1 matrix; that is, a single number.

Referencing Matrix Elements

To reference specific matrix elements, use (row, column) notation. For example:

```
Bonds(1,2)
```

```
ans =
```

```
0.06
```

```
Cash(3)
```

```
ans =
```

```
5280.00
```

You can enlarge matrices using small matrices or vectors as elements. For example,

```
AddBond = [1000 0.065 2];
Bonds = [Bonds; AddBond]
```

adds another row to the matrix and creates

```
Bonds =
```

```
1000 0.06 2
500 0.055 4
1000 0.065 2
```

Likewise,

```
Prices = [987.50
475.00
995.00]
```

```
Bonds = [Prices, Bonds]
```

adds another column and creates

```
Bonds =
```

```
987.50 1000 0.06 2
475.00 500 0.055 4
995.00 1000 0.065 2
```

Finally, the colon (:) is important in generating and referencing matrix elements. For example, to reference the par value, coupon rate, and coupon frequency of the second bond:

```
BondItems = Bonds(2, 2:4)
```

```
BondItems =
```

```
500.00 0.055 4
```

Transposing Matrices

Sometimes matrices are in the wrong configuration for an operation. In MATLAB, the apostrophe or prime character (') transposes a matrix: columns become rows, rows become columns. For example,

```
Cash = [1500 4470 5280 -1299]'
```

produces

```
Cash =
```

```
1500
4470
5280
-1299
```

See Also

More About

- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Matrix Algebra Refresher

In this section...

"Introduction" on page 1-7
 "Adding and Subtracting Matrices" on page 1-7
 "Multiplying Matrices" on page 1-8
 "Dividing Matrices" on page 1-11
 "Solving Simultaneous Linear Equations" on page 1-11
 "Operating Element by Element" on page 1-13

Introduction

The explanations in the sections that follow should help refresh your skills for using matrix algebra and using MATLAB functions.

In addition, *Macro-Investment Analysis* by William Sharpe also provides an excellent explanation of matrix algebra operations using MATLAB. It is available on the web at:

<https://www.stanford.edu/~wfsharpe/mia/mia.htm>

Tip When you are setting up a problem, it helps to "talk through" the units and dimensions associated with each input and output matrix. In the example under "Multiplying Matrices" on page 1-8, one input matrix has five days' closing prices for three stocks, the other input matrix has shares of three stocks in two portfolios, and the output matrix therefore has five days' closing values for two portfolios. It also helps to name variables using descriptive terms.

Adding and Subtracting Matrices

Matrix addition and subtraction operate element-by-element. The two input matrices must have the same dimensions. The result is a new matrix of the same dimensions where each element is the sum or difference of each corresponding input element. For example, consider combining portfolios of different quantities of the same stocks ("shares of stocks A, B, and C [the rows] in portfolios P and Q [the columns] plus shares of A, B, and C in portfolios R and S").

```
Portfolios_PQ = [100    200
                 500    400
                 300    150];
```

```
Portfolios_RS = [175    125
                 200    200
                 100    500];
```

```
NewPortfolios = Portfolios_PQ + Portfolios_RS
```

```
NewPortfolios =
```

```
    275    325
    700    600
    400    650
```

Adding or subtracting a scalar and a matrix is allowed and also operates element-by-element.

```
SmallerPortf = NewPortfolios-10
```

```
SmallerPortf =  
    265.00    315.00  
    690.00    590.00  
    390.00    640.00
```

Multiplying Matrices

Matrix multiplication does *not* operate element-by-element. It operates according to the rules of linear algebra. In multiplying matrices, it helps to remember this key rule: the inner dimensions must be the same. That is, if the first matrix is m -by- 3 , the second must be 3 -by- n . The resulting matrix is m -by- n . It also helps to “talk through” the units of each matrix, as mentioned in “Analyze Sets of Numbers Using Matrix Functions” on page 1-4.

Matrix multiplication also is *not* commutative; that is, it is not independent of order. $A*B$ does *not* equal $B*A$. The dimension rule illustrates this property. If A is 1 -by- 3 matrix and B is 3 -by- 1 matrix, $A*B$ yields a scalar (1 -by- 1) matrix but $B*A$ yields a 3 -by- 3 matrix.

Multiplying Vectors

Vector multiplication follows the same rules and helps illustrate the principles. For example, a stock portfolio has three different stocks and their closing prices today are:

```
ClosePrices = [42.5    15    78.875]
```

The portfolio contains these numbers of shares of each stock.

```
NumShares = [100  
             500  
             300]
```

To find the value of the portfolio, multiply the vectors

```
PortfValue = ClosePrices * NumShares
```

which yields:

```
PortfValue =  
  
    3.5413e+004
```

The vectors are 1 -by- 3 and 3 -by- 1 ; the resulting vector is 1 -by- 1 , a scalar. Multiplying these vectors thus means multiplying each closing price by its respective number of shares and summing the result.

To illustrate order dependence, switch the order of the vectors

```
Values = NumShares * ClosePrices
```

```
Values =  
  
    1.0e+004 *  
  
    0.4250    0.1500    0.7887  
    2.1250    0.7500    3.9438  
    1.2750    0.4500    2.3663
```

which shows the closing values of 100, 500, and 300 shares of each stock, not the portfolio value, and this is meaningless for this example.

Computing Dot Products of Vectors

In matrix algebra, if X and Y are vectors of the same length

$$Y = [y_1, y_2, \dots, y_n]$$

$$X = [x_1, x_2, \dots, x_n]$$

then the dot product

$$X \cdot Y = x_1y_1 + x_2y_2 + \dots + x_ny_n$$

is the scalar product of the two vectors. It is an exception to the commutative rule. To compute the dot product in MATLAB, use `sum(X .* Y)` or `sum(Y .* X)`. Be sure that the two vectors have the same dimensions. To illustrate, use the previous vectors.

```
Value = sum(NumShares .* ClosePrices')
```

```
Value =
```

```
3.5413e+004
```

```
Value = sum(ClosePrices .* NumShares')
```

```
Value =
```

```
3.5413e+004
```

As expected, the value in these cases matches the `PortfValue` computed previously.

Multiplying Vectors and Matrices

Multiplying vectors and matrices follows the matrix multiplication rules and process. For example, a portfolio matrix contains closing prices for a week. A second matrix (vector) contains the stock quantities in the portfolio.

```
WeekClosePr = [42.5      15      78.875
               42.125   15.5     78.75
               42.125   15.125   79
               42.625   15.25    78.875
               43       15.25    78.625];
PortQuan = [100
            500
            300];
```

To see the closing portfolio value for each day, simply multiply

```
WeekPortValue = WeekClosePr * PortQuan
```

```
WeekPortValue =
```

```
1.0e+004 *
```

```
3.5412
```

```
3.5587
```

```

3.5475
3.5550
3.5513

```

The prices matrix is 5-by-3, the quantity matrix (vector) is 3-by-1, so the resulting matrix (vector) is 5-by-1.

Multiplying Two Matrices

Matrix multiplication also follows the rules of matrix algebra. In matrix algebra notation, if A is an m -by- n matrix and B is an n -by- p matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & \cdots & b_{1j} & \cdots & b_{1p} \\ b_{21} & \cdots & b_{2j} & \cdots & b_{2p} \\ \vdots & & \vdots & & \vdots \\ b_{n1} & \cdots & b_{nj} & \cdots & b_{np} \end{bmatrix}$$

then $C = A*B$ is an m -by- p matrix; and the element c_{ij} in the i th row and j th column of C is

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}.$$

To illustrate, assume that there are two portfolios of the same three stocks previously mentioned but with different quantities.

```

Portfolios = [100    200
               500    400
               300    150];

```

Multiplying the 5-by-3 week's closing prices matrix by the 3-by-2 portfolios matrix yields a 5-by-2 matrix showing each day's closing value for both portfolios.

```
PortfolioValues = WeekClosePr * Portfolios
```

```
PortfolioValues =
```

```
1.0e+004 *
```

```

3.5412    2.6331
3.5587    2.6437
3.5475    2.6325
3.5550    2.6456
3.5513    2.6494

```

Monday's values result from multiplying each Monday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. Tuesday's values result from multiplying each Tuesday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. And so on, through the rest of the week. With one simple command, MATLAB quickly performs many calculations.

Multiplying a Matrix by a Scalar

Multiplying a matrix by a scalar is an exception to the dimension and commutative rules. It just operates element-by-element.

```
Portfolios = [100  200
              500  400
              300  150];
```

```
DoublePort = Portfolios * 2
```

```
DoublePort =
    200    400
   1000    800
    600    300
```

Dividing Matrices

Matrix division is useful primarily for solving equations, and especially for solving simultaneous linear equations (see “Solving Simultaneous Linear Equations” on page 1-11). For example, you want to solve for X in $A*X = B$.

In ordinary algebra, you would divide both sides of the equation by A , and X would equal B/A . However, since matrix algebra is not commutative ($A*X \neq X*A$), different processes apply. In formal matrix algebra, the solution involves matrix inversion. MATLAB, however, simplifies the process by providing two matrix division symbols, left and right ($\backslash$ and $/$). In general,

$X = A \backslash B$ solves for X in $A*X = B$ and

$X = B/A$ solves for X in $X*A = B$.

In general, matrix A must be a nonsingular square matrix; that is, it must be invertible and it must have the same number of rows and columns. (Generally, a matrix is invertible if the matrix times its inverse equals the identity matrix. To understand the theory and proofs, consult a textbook on linear algebra such as *Elementary Linear Algebra* by Hill listed in “Bibliography” on page A-2.) MATLAB gives a warning message if the matrix is singular or nearly so.

Solving Simultaneous Linear Equations

Matrix division is especially useful in solving simultaneous linear equations. Consider this problem: Given two portfolios of mortgage-based instruments, each with certain yields depending on the prime rate, how do you weight the portfolios to achieve certain annual cash flows? The answer involves solving two linear equations.

A linear equation is any equation of the form

$$a_1x + a_2y = b,$$

where a_1 , a_2 , and b are constants (with a_1 and a_2 not both 0), and x and y are variables. (It is a linear equation because it describes a line in the xy -plane. For example, the equation $2x + y = 8$ describes a line such that if $x = 2$, then $y = 4$.)

A system of linear equations is a set of linear equations that you usually want to solve at the same time; that is, simultaneously. A basic principle for exact answers in solving simultaneous linear equations requires that there be as many equations as there are unknowns. To get exact answers for x and y , there must be two equations. For example, to solve for x and y in the system of linear equations

$$\begin{aligned} 2x + y &= 13 \\ x - 3y &= -18, \end{aligned}$$

there must be two equations, which there are. Matrix algebra represents this system as an equation involving three matrices: A for the left-side constants, X for the variables, and B for the right-side constants

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 13 \\ -18 \end{bmatrix},$$

where $A \cdot X = B$.

Solving the system simultaneously means solving for X . Using MATLAB,

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix};$$

$$B = \begin{bmatrix} 13 \\ -18 \end{bmatrix};$$

$$X = A \setminus B$$

solves for X in $A \cdot X = B$.

$$X = \begin{bmatrix} 3 & 7 \end{bmatrix}$$

So $x = 3$ and $y = 7$ in this example. In general, you can use matrix algebra to solve any system of linear equations such as

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

by representing them as matrices

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

and solving for X in $A \cdot X = B$.

To illustrate, consider this situation. There are two portfolios of mortgage-based instruments, M1 and M2. They have current annual cash payments of \$100 and \$70 per unit, respectively, based on today's prime rate. If the prime rate moves down one percentage point, their payments would be \$80 and \$40. An investor holds 10 units of M1 and 20 units of M2. The investor's receipts equal cash payments times units, or $R = C \cdot U$, for each prime-rate scenario. As word equations:

	M1	M2
Prime flat:	\$100 * 10 units	+ \$70 * 20 units = \$2400 receipts
Prime down:	\$80 * 10 units	+ \$40 * 20 units = \$1600 receipts

As MATLAB matrices:

```

Cash = [100  70
        80  40];

Units = [10
         20];

Receipts = Cash * Units

Receipts =

    2400
    1600

```

Now the investor asks this question: Given these two portfolios and their characteristics, how many units of each should they hold to receive \$7000 if the prime rate stays flat and \$5000 if the prime drops one percentage point? Find the answer by solving two linear equations.

	M1	M2
Prime flat:	\$100 * x units	+ \$70 * y units = \$7000 receipts
Prime down:	\$80 * x units	+ \$40 * y units = \$5000 receipts

In other words, solve for U (units) in the equation $R(\text{receipts}) = C(\text{cash}) * U(\text{units})$. Using MATLAB left division

```

Cash = [100  70
        80  40];

Receipts = [7000
           5000];

Units = Cash \ Receipts

Units =

    43.7500
    37.5000

```

The investor should hold 43.75 units of portfolio M1 and 37.5 units of portfolio M2 to achieve the annual receipts desired.

Operating Element by Element

Finally, element-by-element arithmetic operations are called operations. To indicate a MATLAB array operation, precede the operator with a period (.). Addition and subtraction, and matrix multiplication and division by a scalar, are already array operations so no period is necessary. When using array operations on two matrices, the dimensions of the matrices must be the same. For example, given vectors of stock dividends and closing prices

```

Dividends = [1.90  0.40  1.56  4.50];
Prices = [25.625  17.75  26.125  60.50];

Yields = Dividends ./ Prices

```

Yields =

0.0741 0.0225 0.0597 0.0744

See Also

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Using Input and Output Arguments with Functions” on page 1-15

Using Input and Output Arguments with Functions

In this section...

"Input Arguments" on page 1-15

"Output Arguments" on page 1-16

Input Arguments

Vector and Matrix Input

By design, MATLAB software can efficiently perform repeated operations on collections of data stored in vectors and matrices. MATLAB code that is written to operate simultaneously on different arrays is said to be vectorized. Vectorized code is not only clean and concise, but is also efficiently processed by MATLAB.

Because MATLAB is optimized for processing vectorized code, many Financial Toolbox functions accept either vector or matrix input arguments, rather than single (scalar) values.

One example of such a function is the `irr` function, which computes the internal rate of return of a cash flow stream. If you input a vector of cash flows from a single cash flow stream, then `irr` returns a scalar rate of return. If you input a matrix of cash flows from multiple cash flow streams, where each matrix column represents a different stream, then `irr` returns a vector of internal rates of return, where the columns correspond to the columns of the input matrix. Many other Financial Toolbox functions work similarly.

As an example, suppose that you make an initial investment of \$100, from which you then receive by a series of annual cash receipts of \$10, \$20, \$30, \$40, and \$50. This cash flow stream is stored in a vector

```
CashFlows = [-100 10 20 30 40 50]'
```

```
CashFlows =
   -100
     10
     20
     30
     40
     50
```

Use the `irr` function to compute the internal rate of return of the cash flow stream.

```
Rate = irr(CashFlows)
```

```
Rate =
    0.1201
```

For the single cash flow stream `CashFlows`, the function returns a scalar rate of return of 0.1201, or 12.01%.

Now, use the `irr` function to compute internal rates of return for multiple cash flow streams.

```
Rate = irr([CashFlows CashFlows CashFlows])
```

```
Rate =  
  
    0.1201    0.1201    0.1201
```

MATLAB performs the same computation on all the assets at once. For the three cash flow streams, the `irr` function returns a vector of three internal rates of return.

In the Financial Toolbox context, vectorized programming is useful in portfolio management. You can organize multiple assets into a single collection by placing data for each asset in a different matrix column or row, then pass the matrix to a Financial Toolbox function.

Character Vector Input

Enter MATLAB character vectors surrounded by single quotes ('character vector').

A character vector is stored as a character array, one ASCII character per element. Thus, the date character vector is

```
DateCharacterVector = '9/16/2017'
```

This date character vector is actually a 1-by-9 vector. If you create a vector or matrix of character vectors, each character vector must have the same length. Using a column vector to create a vector of character vectors can allow you to visually check that all character vectors are the same length. If your character vectors are not the same length, use spaces or zeros to make them the same length, as in the following code.

```
DateFields = ['01/12/2017'  
              '02/14/2017'  
              '03/03/2017'  
              '06/14/2017'  
              '12/01/2017'];
```

`DateFields` is a 5-by-10 array of character vectors.

You cannot mix numbers and character vectors in a vector or matrix. If you input a vector or matrix that contains a mix of numbers and character vectors, MATLAB treats every entry as a character. As an example, input the following code

```
Item = [83  90  99  '14-Sep-1999']
```

```
Item =
```

```
SZc14-Sep-1999
```

The software understands the input not as a 1-by-4 vector, but as a 1-by-14 character array with the value `SZc14-Sep-1999`.

Output Arguments

Some functions return no arguments, some return just one, and some return multiple arguments. Functions that return multiple arguments use the syntax

```
[A, B, C] = function(input_arguments...)
```

to return arguments A, B, and C. If you omit all but one, the function returns the first argument. Thus, for this example if you use the syntax

```
X = function(input_arguments...)
```

the function returns a value for A, but not for B or C.

Some functions that return vectors accept only scalars as arguments. Such functions cannot accept vectors as arguments and return matrices, where each column in the output matrix corresponds to an entry in the input. Output vectors can be variable length.

For example, most functions that require asset life as an input, and return values corresponding to different periods over the asset life, cannot handle vectors or matrices as input arguments. These functions include `amortize`, `depfixdb`, `depgendb`, and `depsoyd`. For example, consider a car for which you want to compute the depreciation schedule. Use the `depfixdb` function to compute a stream of declining-balance depreciation values for the asset. Set the initial value of the asset and the lifetime of the asset. Note that in the returned vector, the asset lifetime determines the number of rows. Now consider a collection of cars with different lifetimes. Because `depfixdb` cannot output a matrix with an unequal number of rows in each column, `depfixdb` cannot accept a single input vector with values for each asset in the collection.

See Also

Related Examples

- “Matrices and Arrays”

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7

Performing Common Financial Tasks

- “Handle and Convert Dates” on page 2-2
- “Analyzing and Computing Cash Flows” on page 2-11
- “Pricing and Computing Yields for Fixed-Income Securities” on page 2-15
- “Treasury Bills Defined” on page 2-25
- “Computing Treasury Bill Price and Yield” on page 2-26
- “Term Structure of Interest Rates” on page 2-29
- “Returns with Negative Prices” on page 2-32
- “Pricing and Analyzing Equity Derivatives” on page 2-39
- “About Life Tables” on page 2-44
- “Case Study for Life Tables Analysis” on page 2-46
- “Machine Learning for Statistical Arbitrage: Introduction” on page 2-48
- “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59
- “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78

Handle and Convert Dates

In this section...
"Date Formats" on page 2-2
"Date Conversions" on page 2-3
"Current Date and Time" on page 2-7
"Determining Specific Dates" on page 2-8
"Determining Holidays" on page 2-8
"Determining Cash-Flow Dates" on page 2-9

Date Formats

Virtually all financial data derives from a time series, functions in Financial Toolbox have extensive date-handling capabilities. The toolbox functions support date or date-and-time formats as character vectors, datetime arrays, or serial date numbers.

- Date character vectors are text that represent date and time, which you can use with multiple formats. For example, 'dd-mmm-yyyy HH:MM:SS', 'dd-mmm-yyyy', and 'mm/dd/yyyy' are all supported text formats for a date character vector. Most often, you work with date character vectors (such as 14-Sep-1999) when dealing with dates.
- Datetime arrays, created using `datetime`, are the best data type for representing points in time. `datetime` values have flexible display formats and up to nanosecond precision, and can account for time zones, daylight saving time, and leap seconds. When `datetime` objects are used as inputs to other Financial Toolbox functions, the format of the input `datetime` object is preserved. For example:

```
originalDate = datetime('now','Format','yyyy-MM-dd HH:mm:ss');  
% Find the next business day  
b = busdate(originalDate)
```

```
b =
```

```
datetime
```

```
2021-05-04 15:59:34
```

- Serial date numbers represent a calendar date as the number of days that have passed since a fixed base date. In MATLAB software, serial date number 1 is January 1,0000 A.D. Financial Toolbox works internally with serial date numbers (such as, 730377). MATLAB also uses serial time to represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so 6:00 p.m. on 14-Sep-1999, in MATLAB, is serial date number 730377.75

Note If you specify a two-digit year, MATLAB assumes that the year lies within the 100-year period centered on the current year. See the function `datenum` for specific information. MATLAB internal date handling and calculations generate no ambiguous values. However, whenever possible, use serial date numbers or date character vectors containing four-digit years.

Many Financial Toolbox functions that require dates as input arguments accept date character vectors, datetime arrays, or serial date numbers. If you are dealing with a few dates at the MATLAB command-line level, date character vectors are more convenient. If you are using Financial Toolbox functions on large numbers of dates, as in analyzing large portfolios or cash flows, performance improves if you use datetime arrays or serial date numbers. For more information, see “Represent Dates and Times in MATLAB”.

Date Conversions

Financial Toolbox provides functions that convert date character vectors to or from serial date numbers. In addition, you can convert character vectors or serial date numbers to datetime arrays.

Functions that convert between date formats are:

<code>datedisp</code>	Displays a numeric matrix with date entries formatted as date character vectors.
<code>datenum</code>	Converts a date character vector to a serial date number.
<code>datestr</code>	Converts a serial date number to a date character vector.
<code>datetime</code>	Converts from date character vectors or serial date numbers to create a datetime array.
<code>datevec</code>	Converts a serial date number or date character vector to a date vector whose elements are [Year Month Day Hour Minute Second].
<code>m2xdate</code>	Converts MATLAB serial date number to Excel® serial date number.
<code>x2mdate</code>	Converts Microsoft® Excel serial date number to MATLAB serial date number.

For more information, see “Convert Between Text and datetime or duration Values”.

Convert Between Datetime Arrays and Character Vectors

A date can be a character vector composed of fields related to a specific date and time. There are several ways to represent dates and times in several text formats. For example, all the following are character vectors represent August 23, 2010 at 04:35:42 PM:

```
'23-Aug-2010 04:35:06 PM'
'Wednesday, August 23'
'08/23/10 16:35'
'Aug 23 16:35:42.946'
```

A date character vector includes characters that separate the fields, such as the hyphen, space, and colon used here:

```
d = '23-Aug-2010 16:35:42'
```

Convert one or more date character vectors to a `datetime` array using the `datetime` function. For the best performance, specify the format of the input character vectors as an input to `datetime`.

Note The specifiers that `datetime` uses to describe date and time formats differ from the specifiers that the `datestr`, `datevec`, and `datenum` functions accept.

```
t = datetime(d, 'InputFormat', 'dd-MMM-yyyy HH:mm:ss')
t =
    23-Aug-2010 16:35:42
```

Although the date string, `d`, and the `datetime` scalar, `t`, look similar, they are not equal. View the size and data type of each variable.

```
whos d t
```

Name	Size	Bytes	Class	Attributes
d	1x20	40	char	
t	1x1	121	datetime	

Convert a `datetime` array to a character vector that uses `char` or `cellstr`. For example, convert the current date and time to a timestamp to append to a file name.

```
t = datetime('now', 'Format', 'yyyy-MM-dd' 'T' 'HHmmss')
t =
    datetime
    2016-12-11T125628
S = char(t);
filename = ['myTest_', S]
filename =
    'myTest_2016-12-11T125628'
```

Convert Serial Date Numbers to Datetime Arrays

Serial time can represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so the character vector '31-Oct-2003, 6:00 PM' in MATLAB is date number 731885.75.

Convert one or more serial date numbers to a `datetime` array using the `datetime` function. Specify the type of date number that is being converted:

```
t = datetime(731885.75, 'ConvertFrom', 'datenum')
t =
    datetime
    31-Oct-2003 18:00:00
```

Convert Datetime Arrays to Numeric Values

Some MATLAB functions accept numeric data types but not `datetime` values as inputs. To apply these functions to your date and time data, first, convert `datetime` values to meaningful numeric values, and then call the function. For example, the `log` function accepts `double` inputs but not `datetime` inputs. Suppose that you have a `datetime` array of dates spanning the course of a research study or experiment.


```
t = datetime(2014,6,18) + calmonths(1:4)
t =
    1×4 datetime array

    18-Jul-2014    18-Aug-2014    18-Sep-2014    18-Oct-2014
```

Subtract the origin value. For example, the origin value can be the starting day of an experiment.

```
dt = t - datetime(2014,7,1)
dt =
    1×4 duration array

    408:00:00    1152:00:00    1896:00:00    2616:00:00
```

`dt` is a `duration` array. Convert `dt` to a double array of values in units of years, days, hours, minutes, or seconds by using the `years`, `days`, `hours`, `minutes`, or `seconds` function, respectively.

```
x = hours(dt)
x =
         408         1152         1896         2616
```

Pass the double array as the input to the `log` function.

```
y = log(x)
y =
    6.0113    7.0493    7.5475    7.8694
```

Input Conversions with `datenum`

The `datenum` function is important for using Financial Toolbox software efficiently. `datenum` takes an input date character vector in any of several formats, with `'dd-mmm-yyyy'`, `'mm/dd/yyyy'`, or `'dd-mmm-yyyy, hh:mm:ss.ss'` formats being the most common. The input date character vector can have up to six fields formed by letters and numbers separated by any other characters, such that:

- The day field is an integer from 1 through 31.
- The month field is either an integer from 1 through 12 or an alphabetical character vector with at least three characters.
- The year field is a nonnegative integer. If only two numbers are specified, then the year is assumed to lie within the 100-year period centered on the current year. If the year is omitted, the current year is the default.
- The hours, minutes, and seconds fields are optional. They are integers separated by colons or followed by `'am'` or `'pm'`.

For example, if the current year is 1999, then all these dates are equivalent:

```
'17-May-1999'
'17-May-99'
'17-may'
'May 17, 1999'
```

```
'5/17/99'  
'5/17'
```

Also, both of these formats represent the same time.

```
'17-May-1999, 18:30'  
'5/17/99/6:30 pm'
```

The default format for numbers-only input follows the US convention. Therefore, 3/6 is March 6, not June 3.

With `datenum`, you can convert dates into serial date format, store them in a matrix variable, and then later pass the variable to a function. Alternatively, you can use `datenum` directly in a function input argument list.

For example, consider the function `bndprice` that computes the price of a bond given the yield to maturity. First set up variables for the yield to maturity, coupon rate, and the necessary dates.

```
Yield      = 0.07;  
CouponRate = 0.08;  
Settle     = datenum('17-May-2000');  
Maturity   = datenum('01-Oct-2000');
```

Then call the function with the variables.

```
bndprice(Yield,CouponRate,Settle,Maturity)
```

```
ans =  
  
100.3503
```

Alternatively, convert date character vectors to serial date numbers directly in the function input argument list.

```
bndprice(0.07,0.08,datenum('17-May-2000'),...  
datenum('01-Oct-2000'))
```

```
ans =  
  
100.3503
```

`bndprice` is an example of a function designed to detect the presence of date character vectors and make the conversion automatically. For functions like `bndprice`, date character vectors can be passed directly.

```
bndprice(0.07,0.08,'17-May-2000','01-Oct-2000')
```

```
ans =  
  
100.3503
```

The decision to represent dates as either date character vectors or serial date numbers is often a matter of convenience. For example, when formatting data for visual display or for debugging date-handling code, you can view dates more easily as date character vectors because serial date numbers are difficult to interpret. Alternately, serial date numbers are just another type of numeric data, which you can place in a matrix along with any other numeric data for convenient manipulation.

Remember that if you create a vector of input date character vectors, use a column vector, and be sure that all character vectors are the same length. To ensure that the character vectors are the same

length, fill the character vectors with spaces or zeros. For more information, see “Character Vector Input” on page 1-16.

Output Conversions with `datestr`

The `datestr` function converts a serial date number to one of 19 different date character vector output formats showing date, time, or both. The default output for dates is a day-month-year character vector, for example, 24-Aug-2000. The `datestr` function is useful for preparing output reports.

datestr Format	Description
01-Mar-2000 15:45:17	day-month-year hour:minute:second
01-Mar-2000	day-month-year
03/01/00	month/day/year
Mar	month, three letters
M	month, single letter
3	month number
03/01	month/day
1	day of month
Wed	day of week, three letters
W	day of week, single letter
2000	year, four numbers
99	year, two numbers
Mar01	month year
15:45:17	hour:minute:second
03:45:17 PM	hour:minute:second AM or PM
15:45	hour:minute
03:45 PM	hour:minute AM or PM
Q1-99	calendar quarter-year
Q1	calendar quarter

Current Date and Time

The `today` and `now` functions return serial date numbers for the current date, and the current date and time, respectively.

`today`

`ans =`

736675

`now`

`ans =`

7.3668e+05

The MATLAB function `date` returns a character vector for the current date.

```
date
ans =
    '11-Dec-2016'
```

Determining Specific Dates

Financial Toolbox provides many functions for determining specific dates. For example, assume that you schedule an accounting procedure for the last Friday of every month. Use the `lweekdate` function to return those dates for the year 2000. The input argument 6 specifies Friday.

```
Fridates = lweekdate(6,2000,1:12);
Fridays = datestr(Fridates)
```

```
Fridays =
    12×11 char array
    '28-Jan-2000'
    '25-Feb-2000'
    '31-Mar-2000'
    '28-Apr-2000'
    '26-May-2000'
    '30-Jun-2000'
    '28-Jul-2000'
    '25-Aug-2000'
    '29-Sep-2000'
    '27-Oct-2000'
    '24-Nov-2000'
    '29-Dec-2000'
```

Another example of needing specific dates could be that your company closes on Martin Luther King Jr. Day, which is the third Monday in January. You can use the `nweekdate` function to determine those specific dates for 2011 through 2014.

```
MLKDates = nweekdate(3,2,2011:2014,1);
MLKDays = datestr(MLKDates)
```

```
MLKDays =
    4×11 char array
    '17-Jan-2011'
    '16-Jan-2012'
    '21-Jan-2013'
    '20-Jan-2014'
```

Determining Holidays

Accounting for holidays and other nontrading days is important when you examine financial dates. Financial Toolbox provides the `holidays` function, which contains holidays and special nontrading days for the New York Stock Exchange from 1950 through 2030, inclusive. In addition, you can use `nyseclosures` to evaluate all known or anticipated closures of the New York Stock Exchange from

January 1, 1885, to December 31, 2050. `nyseclosures` returns a vector of serial date numbers corresponding to market closures between the dates `StartDate` and `EndDate`, inclusive.

In this example, use `holidays` to determine the standard holidays in the last half of 2012.

```
LHHDates = holidays('1-Jul-2012','31-Dec-2012');
LHHDays = datestr(LHHDates)
```

```
LHHDays =
```

```
6×11 char array
```

```
'04-Jul-2012'
'03-Sep-2012'
'29-Oct-2012'
'30-Oct-2012'
'22-Nov-2012'
'25-Dec-2012'
```

You can then use the `busdate` function to determine the next business day in 2012 after these holidays.

```
LHNextDates = busdate(LHHDates);
LHNextDays = datestr(LHNextDates)
```

```
LHNextDays =
```

```
6×11 char array
```

```
'05-Jul-2012'
'04-Sep-2012'
'31-Oct-2012'
'31-Oct-2012'
'23-Nov-2012'
'26-Dec-2012'
```

Determining Cash-Flow Dates

To determine cash-flow dates for securities with periodic payments, use `cfdates`. This function accounts for the coupons per year, the day-count basis, and the end-of-month rule. For example, you can determine the cash-flow dates for a security that pays four coupons per year on the last day of the month using an `actual/365` day-count basis. To do so, enter the settlement date, the maturity date, and the parameters for `Period`, `Basis`, and `EndMonthRule`.

```
PayDates = cfdates('14-Mar-2000','30-Nov-2001',4,3,1);
PayDays = datestr(PayDates)
```

```
PayDays =
```

```
7×11 char array
```

```
'31-May-2000'
'31-Aug-2000'
'30-Nov-2000'
'28-Feb-2001'
'31-May-2001'
'31-Aug-2001'
'30-Nov-2001'
```

See Also

`datedisp` | `datenum` | `datestr` | `datetime` | `datevec` | `format` | `date` | `holidays` | `nyseclosures` | `busdate` | `cfdates` | `addBusinessCalendar`

Related Examples

- “Convert Between Text and datetime or duration Values”
- “Read Collection or Sequence of Spreadsheet Files”
- “Trading Calendars User Interface” on page 12-2
- “UICalendar User Interface” on page 12-4

More About

- “Convert Dates Between Microsoft Excel and MATLAB” (Spreadsheet Link)

External Websites

- Automated Data Cleaning and Preparation in MATLAB (43 min)

Analyzing and Computing Cash Flows

In this section...

"Introduction" on page 2-11
 "Interest Rates/Rates of Return" on page 2-11
 "Present or Future Values" on page 2-12
 "Depreciation" on page 2-13
 "Annuities" on page 2-13

Introduction

Financial Toolbox cash-flow functions compute interest rates and rates of return, present or future values, depreciation streams, and annuities.

Some examples in this section use this income stream: an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...
          9500, 9500, 9500];
```

Interest Rates/Rates of Return

This example shows how to compute the internal rate of return of the cash stream using `irr`.

Specify the income stream as an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...
          9500, 9500, 9500];
```

Use `irr` to compute the internal rate of return of the cash stream.

```
ROR = irr(Stream)
```

```
ROR =
0.1172
```

The rate of return is 11.72%.

The internal rate of return of a cash flow may not have a unique value. Every time the sign changes in a cash flow, the equation defining `irr` can give up to two additional answers. An `irr` computation requires solving a polynomial equation, and the number of real roots of such an equation can depend on the number of sign changes in the coefficients. The equation for internal rate of return is

$$\frac{cf_1}{(1+r)} + \frac{cf_2}{(1+r)^2} + \dots + \frac{cf_n}{(1+r)^n} + Investment = 0,$$

where *Investment* is a (negative) initial cash outlay at time 0, *cf_n* is the cash flow in the *n*th period, and *n* is the number of periods. `irr` finds the rate *r* such that the present value of the cash flow

equals the initial investment. If all the cf_n s are positive there is only one solution. Every time there is a change of sign between coefficients, up to two additional real roots are possible.

Another toolbox rate function, `effrr`, calculates the effective rate of return given an annual interest rate (also known as nominal rate or annual percentage rate, APR) and number of compounding periods per year. To find the effective rate of a 9% APR compounded monthly, enter

```
Rate = effrr(0.09, 12)
```

```
Rate =  
0.0938
```

The Rate is 9.38%.

A companion function `nomrr` computes the nominal rate of return given the effective annual rate and the number of compounding periods.

Present or Future Values

This example shows how to compute the present or future value of cash flows at regular or irregular time intervals with equal or unequal payments.

To compute the present or future value, you can use the following functions: `fvfix`, `fvvar`, `pvfix`, and `pvvar`. The `-fix` functions assume equal cash flows at regular intervals, while the `-var` functions allow irregular cash flows at irregular periods.

Specify the income stream as an initial investment of \$20,000 followed by three annual return payments, a second investment of \$5,000, then four more returns. Investments are negative cash flows, return payments are positive cash flows.

```
Stream = [-20000, 2000, 2500, 3500, -5000, 6500, ...  
          9500, 9500, 9500];
```

Use `irr` to compute the internal rate of return of the cash stream.

```
ROR = irr(Stream)
```

```
ROR =  
0.1172
```

Compute the net present value of the sample income stream for which you computed the internal rate of return. This exercise also serves as a check on that calculation because the net present value of a cash stream at its internal rate of return should be zero. Enter

```
NPV = pvvar(Stream, ROR)
```

```
NPV =  
5.9117e-12
```

The NPV is very close to zero. The answer usually is not *exactly* zero due to rounding errors and the computational precision of the computer. Note, other toolbox functions behave similarly. The functions that compute a bond's yield, for example, often must solve a nonlinear equation. If you then use that yield to compute the net present value of the bond's income stream, it usually does not *exactly* equal the purchase price, but the difference is negligible for practical applications.

Depreciation

This example shows how to compute standard depreciation schedules using `depgendb`.

The following code depreciates an automobile worth \$15,000 over five years with a salvage value of \$1,500. It computes the general declining balance using two different depreciation rates: 50% (or 1.5), and 100% (or 2.0, also known as double declining balance).

```
Decline1 = depgendb(15000, 1500, 5, 1.5)
```

```
Decline1 = 1×5  
103 ×
```

```
4.5000    3.1500    2.2050    1.5435    2.1015
```

```
Decline2 = depgendb(15000, 1500, 5, 2.0)
```

```
Decline2 = 1×5  
103 ×
```

```
6.0000    3.6000    2.1600    1.2960    0.4440
```

These results indicate the actual depreciation amount for the first four years and the remaining depreciable value as the entry for the fifth year.

Annuities

This example shows how to work with annuities using `annurate`.

The following code shows how to compute the interest rate associated with a series of loan payments when only the payment amounts and principal are known. For a loan whose original value was \$5000.00 and which was paid back monthly over four years at \$130.00/month:

```
Rate = annurate(4*12, 130, 5000, 0, 0)
```

```
Rate =  
0.0094
```

The function returns a rate of 0.0094 monthly, or about 11.28% annually.

You can use a present-value function (`pvinfos`) to compute the initial principal when the payment and rate are known. For a loan paid at \$300.00/month over four years at 11% annual interest:

```
Principal = pvfinfo(0.11/12, 4*12, 300, 0, 0)
```

```
Principal =  
1.1607e+04
```

The function returns the original principal value of \$11,607.43.

You can compute an amortization schedule using `amortize` for a loan or annuity. For example, the original value was \$5000.00 and was paid back over 12 months at an annual rate of 9%.

```
[Prpmt, Intpmt, Balance, Payment] = amortize(0.09/12, 12, 5000, 0, 0)
```

```
Prpmt = 1×12
```

```
399.7574 402.7556 405.7762 408.8196 411.8857 414.9748 418.0872 421.2228 424.3820 427.5
```

```
Intpmt = 1×12
```

```
37.5000 34.5018 31.4812 28.4378 25.3717 22.2825 19.1702 16.0346 12.8754 9.
```

```
Balance = 1×12  
103 ×
```

```
4.6002 4.1975 3.7917 3.3829 2.9710 2.5560 2.1379 1.7167 1.2923 0.
```

```
Payment =  
437.2574
```

See Also

`irr` | `effrr` | `nomrr` | `fvfix` | `fvvar` | `pvfix` | `pvvar`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Pricing and Computing Yields for Fixed-Income Securities” on page 2-15

Pricing and Computing Yields for Fixed-Income Securities

In this section...

“Introduction” on page 2-15
 “Fixed-Income Terminology” on page 2-15
 “Framework” on page 2-18
 “Default Parameter Values” on page 2-18
 “Coupon Date Calculations” on page 2-20
 “Yield Conventions” on page 2-21
 “Pricing Functions” on page 2-21
 “Yield Functions” on page 2-21
 “Fixed-Income Sensitivities” on page 2-22

Introduction

The Financial Toolbox product provides functions for computing accrued interest, price, yield, convexity, and duration of fixed-income securities. Various conventions exist for determining the details of these computations. The Financial Toolbox software supports conventions specified by the Securities Industry and Financial Markets Association (SIFMA), used in the US markets, the International Capital Market Association (ICMA), used mainly in the European markets, and the International Swaps and Derivatives Association (ISDA). For historical reasons, SIFMA is referred to in Financial Toolbox documentation as SIA and ISMA is referred to as International Capital Market Association (ICMA). Financial Instruments Toolbox™ supports additional functionality for pricing fixed-income securities. For more information, see “Price Interest-Rate Instruments” (Financial Instruments Toolbox).

Fixed-Income Terminology

Since terminology varies among texts on this subject, here are some basic definitions that apply to these Financial Toolbox functions.

The *settlement date* of a bond is the date when money first changes hands; that is, when a buyer pays for a bond. It need not coincide with the *issue date*, which is the date a bond is first offered for sale.

The *first coupon date* and *last coupon date* are the dates when the first and last coupons are paid, respectively. Although bonds typically pay periodic annual or semiannual coupons, the length of the first and last coupon periods may differ from the standard coupon period. The toolbox includes price and yield functions that handle these odd first and/or last periods.

Successive *quasi-coupon dates* determine the length of the standard coupon period for the fixed income security of interest, and do not necessarily coincide with actual coupon payment dates. The toolbox includes functions that calculate both actual and quasi-coupon dates for bonds with odd first and/or last periods.

Fixed-income securities can be purchased on dates that do not coincide with coupon payment dates. In this case, the bond owner is not entitled to the full value of the coupon for that period. When a bond is purchased between coupon dates, the buyer must compensate the seller for the pro-rata share of the coupon interest earned from the previous coupon payment date. This pro-rata share of

the coupon payment is called *accrued interest*. The *purchase price*, the price paid for a bond, is the quoted market price plus accrued interest.

The *maturity date* of a bond is the date when the issuer returns the final face value, also known as the *redemption value* or *par value*, to the buyer. The *yield-to-maturity* of a bond is the nominal compound rate of return that equates the present value of all future cash flows (coupons and principal) to the current market price of the bond.

Period

The period of a bond refers to the frequency with which the issuer of a bond makes coupon payments to the holder.

Period of a Bond

Period Value	Payment Schedule
0	No coupons (Zero coupon bond)
1	Annual
2	Semiannual
3	Tri-annual
4	Quarterly
6	Bi-monthly
12	Monthly

Basis

The basis of a bond refers to the basis or day-count convention for a bond. Day count basis determines how interest accrues over time for various instruments and the amount transferred on interest payment dates. Basis is normally expressed as a fraction in which the numerator determines the number of days between two dates, and the denominator determines the number of days in the year.

For example, the numerator of *actual/actual* means that when determining the number of days between two dates, count the actual number of days; the denominator means that you use the actual number of days in the given year in any calculations (either 365 or 366 days depending on whether the given year is a leap year). The calculation of accrued interest for dates between payments also uses day count basis. Day count basis is a fraction of **Number of interest accrual days / Days in the relevant coupon period**.

Note Although the concept of day count sounds deceptively simple, the actual calculation of day counts can be complex. You can find a good discussion of day counts and the formulas for calculating them in Chapter 5 of Stigum and Robinson, *Money Market and Bond Calculations* in “Bibliography” on page A-2. For more information on Basis, see EMU and Market Conventions: Recent Developments.

Supported day count conventions and basis values are:

Basis Value	Day Count Convention
0	actual/actual (default) — Number of days in both a period and a year is the actual number of days. Also, another common actual/actual basis is basis 12.
1	30/360 SIA — Year fraction is calculated based on a 360 day year with 30-day months, after applying the following rules: If the first date and the second date are the last day of February, the second date is changed to the 30th. If the first date falls on the 31st or is the last day of February, it is changed to the 30th. If after the preceding test, the first day is the 30th and the second day is the 31st, then the second day is changed to the 30th.
2	actual/360 — Number of days in a period is equal to the actual number of days, however the number of days in a year is 360.
3	actual/365 — Number of days in a period is equal to the actual number of days, however the number of days in a year is 365 (even in a leap year).
4	30/360 PSA — Number of days in every month is set to 30 (including February). If the start date of the period is either the 31st of a month or the last day of February, the start date is set to the 30th, while if the start date is the 30th of a month and the end date is the 31st, the end date is set to the 30th. The number of days in a year is 360.
5	30/360 ISDA — Number of days in every month is set to 30, except for February where it is the actual number of days. If the start date of the period is the 31st of a month, the start date is set to the 30th while if the start date is the 30th of a month and the end date is the 31st, the end date is set to the 30th. The number of days in a year is 360.
6	30E /360 — Number of days in every month is set to 30 except for February where it is equal to the actual number of days. If the start date or the end date of the period is the 31st of a month, that date is set to the 30th. The number of days in a year is 360.
7	actual/365 Japanese — Number of days in a period is equal to the actual number of days, except for leap days (29th February) which are ignored. The number of days in a year is 365 (even in a leap year).
8	actual/actual ICMA — Number of days in both a period and a year is the actual number of days and the compounding frequency is annual.
9	actual/360 ICMA — Number of days in a period is equal to the actual number of days, however the number of days in a year is 360 and the compounding frequency is annual.
10	actual/365 ICMA — Number of days in a period is equal to the actual number of days, however the number of days in a year is 365 (even in a leap year) and the compounding frequency is annual.
11	30/360 ICMA — Number of days in every month is set to 30, except for February where it is equal to the actual number of days. If the start date or the end date of the period is the 31st of a month, that date is set to the 30th. The number of days in a year is 360 and the compounding frequency is annual.
12	actual/365 ISDA — The day count fraction is calculated using the following formula: (Actual number of days in period that fall in a leap year / 366) + (Actual number of days in period that fall in a normal year / 365). Basis 12 is also referred to as actual/actual ISDA.
13	bus/252 — The number of days in a period is equal to the actual number of business days. The number of business days in a year is 252.

End-of-Month Rule

The *end-of-month rule* affects a bond's coupon payment structure. When the rule is in effect, a security that pays a coupon on the last actual day of a month will always pay coupons on the last day of the month. This means, for example, that a semiannual bond that pays a coupon on February 28 in nonleap years will pay coupons on August 31 in all years and on February 29 in leap years.

End-of-Month Rule

End-of-Month Rule Value	Meaning
1 (default)	Rule in effect.
0	Rule not in effect.

Framework

Although not all Financial Toolbox functions require the same input arguments, they all accept the following common set of input arguments.

Common Input Arguments

Input	Meaning
Settle	Settlement date
Maturity	Maturity date
Period	Coupon payment period
Basis	Day-count basis
EndMonthRule	End-of-month payment rule
IssueDate	Bond issue date
FirstCouponDate	First coupon payment date
LastCouponDate	Last coupon payment date

Of the common input arguments, only `Settle` and `Maturity` are required. All others are optional. They are set to the default values if you do not explicitly set them. By default, the `FirstCouponDate` and `LastCouponDate` are nonapplicable. In other words, if you do not specify `FirstCouponDate` and `LastCouponDate`, the bond is assumed to have no odd first or last coupon periods. In this case, the bond is a standard bond with a coupon payment structure based solely on the maturity date.

Default Parameter Values

To illustrate the use of default values in Financial Toolbox functions, consider the `cfdates` function, which computes actual cash flow payment dates for a portfolio of fixed income securities regardless of whether the first and/or last coupon periods are normal, long, or short.

The complete calling syntax with the full input argument list is

```
CFlowDates = cfdates(Settle, Maturity, Period, Basis, ...
    EndMonthRule, IssueDate, FirstCouponDate, LastCouponDate)
```

while the minimal calling syntax requires only settlement and maturity dates

```
CFlowDates = cfdates(Settle, Maturity)
```

Single Bond Example

As an example, suppose that you have a bond with these characteristics:

```
Settle      = '20-Sep-1999'
Maturity    = '15-Oct-2007'
Period      = 2
Basis       = 0
EndMonthRule = 1
IssueDate   = NaN
FirstCouponDate = NaN
LastCouponDate = NaN
```

Period, Basis, and EndMonthRule are set to their default values, and IssueDate, FirstCouponDate, and LastCouponDate are set to NaN.

Formally, a NaN is an IEEE® arithmetic standard for *Not-a-Number* and is used to indicate the result of an undefined operation (for example, zero divided by zero). However, NaN is also a convenient placeholder. In the SIA functions of Financial Toolbox software, NaN indicates the presence of a nonapplicable value. It tells the Financial Toolbox functions to ignore the input value and apply the default. Setting IssueDate, FirstCouponDate, and LastCouponDate to NaN in this example tells cfdates to assume that the bond has been issued before settlement and that no odd first or last coupon periods exist.

Having set these values, all these calls to cfdates produce the same result.

```
cfdates(Settle, Maturity)
cfdates(Settle, Maturity, Period)
cfdates(Settle, Maturity, Period, [])
cfdates(Settle, Maturity, [], Basis)
cfdates(Settle, Maturity, [], [])
cfdates(Settle, Maturity, Period, [], EndMonthRule)
cfdates(Settle, Maturity, Period, [], NaN)
cfdates(Settle, Maturity, Period, [], [], IssueDate)
cfdates(Settle, Maturity, Period, [], [], IssueDate, [], [])
cfdates(Settle, Maturity, Period, [], [], [], [], LastCouponDate)
cfdates(Settle, Maturity, Period, Basis, EndMonthRule, ...
IssueDate, FirstCouponDate, LastCouponDate)
```

Thus, leaving a particular input unspecified has the same effect as passing an empty matrix ([]) or passing a NaN - all three tell cfdates (and other Financial Toolbox functions) to use the default value for a particular input parameter.

Bond Portfolio Example

Since the previous example included only a single bond, there was no difference between passing an empty matrix or passing a NaN for an optional input argument. For a portfolio of bonds, however, using NaN as a placeholder is the only way to specify default acceptance for some bonds while explicitly setting nondefault values for the remaining bonds in the portfolio.

Now suppose that you have a portfolio of two bonds.

```
Settle      = '20-Sep-1999'
Maturity    = ['15-Oct-2007'; '15-Oct-2010']
```

These calls to cfdates all set the coupon period to its default value (Period = 2) for both bonds.

```
cfdates(Settle, Maturity, 2)
cfdates(Settle, Maturity, [2 2])
cfdates(Settle, Maturity, [])
cfdates(Settle, Maturity, NaN)
cfdates(Settle, Maturity, [NaN NaN])
cfdates(Settle, Maturity)
```

The first two calls explicitly set `Period = 2`. Since `Maturity` is a 2-by-1 vector of maturity dates, `cfdates` knows that you have a two-bond portfolio.

The first call specifies a single (that is, scalar) 2 for `Period`. Passing a scalar tells `cfdates` to apply the scalar-valued input to all bonds in the portfolio. This is an example of implicit scalar-expansion. The settlement date has been implicit scalar-expanded as well.

The second call also applies the default coupon period by explicitly passing a two-element vector of 2's. The third call passes an empty matrix, which `cfdates` interprets as an invalid period, for which the default value is used. The fourth call is similar, except that a `NaN` has been passed. The fifth call passes two `NaN`'s, and has the same effect as the third. The last call passes the minimal input set.

Finally, consider the following calls to `cfdates` for the same two-bond portfolio.

```
cfdates(Settle, Maturity, [4 NaN])
cfdates(Settle, Maturity, [4 2])
```

The first call explicitly sets `Period = 4` for the first bond and implicitly sets the default `Period = 2` for the second bond. The second call has the same effect as the first but explicitly sets the periodicity for both bonds.

The optional input `Period` has been used for illustrative purpose only. The default-handling process illustrated in the examples applies to any of the optional input arguments.

Coupon Date Calculations

Calculating coupon dates, either actual or quasi dates, is notoriously complicated. Financial Toolbox software follows the SIA conventions in coupon date calculations.

The first step in finding the coupon dates associated with a bond is to determine the reference, or synchronization date (the *sync date*). Within the SIA framework, the order of precedence for determining the sync date is:

- 1 The first coupon date
- 2 The last coupon date
- 3 The maturity date

In other words, a Financial Toolbox function first examines the `FirstCouponDate` input. If `FirstCouponDate` is specified, coupon payment dates and quasi-coupon dates are computed with respect to `FirstCouponDate`; if `FirstCouponDate` is unspecified, empty (`[]`), or `NaN`, then the `LastCouponDate` is examined. If `LastCouponDate` is specified, coupon payment dates and quasi-coupon dates are computed with respect to `LastCouponDate`. If both `FirstCouponDate` and `LastCouponDate` are unspecified, empty (`[]`), or `NaN`, the `Maturity` (a required input argument) serves as the synchronization date.

Yield Conventions

There are two yield and time factor conventions that are used in the Financial Toolbox software – these are determined by the input `basis`. Specifically, bases 0 to 7 are assumed to have semiannual compounding, while bases 8 to 12 are assumed to have annual compounding regardless of the period of the bond's coupon payments (including zero-coupon bonds). In addition, any yield-related sensitivity (that is, duration and convexity), when quoted on a periodic basis, follows this same convention. (See `bndconvp`, `bndconvy`, `bnddurp`, `bnddury`, and `bndkrdur`.)

Pricing Functions

This example shows how to compute the price of a bond with an odd first period using `bndprice`.

Assume that you have a bond with these characteristics:

```
Settle          = '11-Nov-1992';
Maturity        = '01-Mar-2005';
IssueDate       = '15-Oct-1992';
FirstCouponDate = '01-Mar-1993';
CouponRate      = 0.0785;
Yield           = 0.0625;
```

Allow coupon payment period (`Period = 2`), day-count basis (`Basis = 0`), and end-of-month rule (`EndMonthRule = 1`) to assume the default values. Also, assume that there is no odd last coupon date and that the face value of the bond is \$100. Calling the function:

```
[Price, AccruedInt] = bndprice(Yield, CouponRate, Settle, ...
Maturity, [], [], [], IssueDate, FirstCouponDate)
```

```
Price =
113.5977
```

```
AccruedInt =
0.5855
```

`bndprice` returns a price of \$113.60 and accrued interest of \$0.59.

Note, `bndprice` uses nonlinear formulas to compute the price of a security. For this reason, Financial Toolbox™ software uses Newton's method when solving for an independent variable within a formula.

Yield Functions

This example shows how to use `bndyield` compute the yield of a bond that has odd first and last periods and settlement in the first period.

Set up variables for settlement, maturity date, issue, first coupon, and a last coupon date.

```
Settle          = '12-Jan-2000';
Maturity        = '01-Oct-2001';
IssueDate       = '01-Jan-2000';
FirstCouponDate = '15-Jan-2000';
LastCouponDate  = '15-Apr-2000';
```

Assume a face value of \$100. Specify a purchase price of \$95.70, a coupon rate of 4%, quarterly coupon payments, and a 30/360 day-count convention (`Basis = 1`).

```
Price      = 95.7;
CouponRate = 0.04;
Period     = 4;
Basis      = 1;
EndMonthRule = 1;
```

Call the `bndyield` function.

```
Yield = bndyield(Price, CouponRate, Settle, Maturity, Period, ...
Basis, EndMonthRule, IssueDate, FirstCouponDate, LastCouponDate)

Yield =
0.0659
```

The function returns a `Yield = 0.0659` (6.60%).

Fixed-Income Sensitivities

Financial Toolbox software supports the following options for managing interest-rate risk for one or more bonds:

- `bnddurp` and `bnddury` support duration and convexity analysis based on market quotes and assume parallel shifts in the bond yield curve.
- `bndkrdur` supports key rate duration based on a market yield curve and can model nonparallel shifts in the bond yield curve.

Calculating Duration and Convexity for Bonds

This example shows how to compute the annualized Macaulay and modified durations and the periodic Macaulay duration for a bond.

The Macaulay duration of an income stream, such as a coupon bond, measures how long, on average, the owner waits before receiving a payment. It is the weighted average of the times payments are made, with the weights at time T equal to the present value of the money received at time T . The modified duration is the Macaulay duration discounted by the per-period interest rate; that is, divided by $(1 + \text{rate}/\text{frequency})$. The *Macaulay duration* is a measure of price sensitivity to yield changes. This duration is measured in years and is a weighted average-time-to-maturity of an instrument.

To illustrate, the following code computes the annualized Macaulay and modified durations and the periodic Macaulay duration for a bond with settlement (12-Jan-2000) and maturity (01-Oct-2001) dates, a 5% coupon rate, and a 4.5% yield to maturity. For simplicity, any optional input arguments assume default values (that is, semiannual coupons, and day-count basis = 0 (actual/actual), coupon payment structure synchronized to the maturity date, and end-of-month payment rule in effect).

```
CouponRate = 0.05;
Yield = 0.045;
Settle = datetime(2000,1,12);
Maturity = datetime(2001,10,1);

[ModDuration, YearDuration, PerDuration] = bnddury(Yield, ...
CouponRate, Settle, Maturity)
```

```
ModDuration =
1.6107
```

```
YearDuration =
1.6470
```

```
PerDuration =
3.2940
```

The durations are:

- `ModDuration` = 1.6107 (years)
- `YearDuration` = 1.6470 (years)
- `PerDuration` = 3.2940 (semiannual periods)

Note that the semiannual periodic Macaulay duration (`PerDuration`) is twice the annualized Macaulay duration (`YearDuration`).

Calculating Key Rate Durations for Bonds

This example shows how to compute the key rate duration of the US Treasury Bond.

Key rate duration enables you to evaluate the sensitivity and price of a bond to nonparallel changes in the spot or zero curve by decomposing the interest rate risk along the spot or zero curve. Key rate duration refers to the process of choosing a set of key rates and computing a duration for each rate. Specifically, for each key rate, while the other rates are held constant, the key rate is shifted up and down (and intermediate cash flow dates are interpolated), and then the present value of the security given the shifted curves is computed.

The calculation of `bndkrdur` supports

$$krdur_i = \frac{(PV_{down} - PV_{up})}{(PV \times ShiftValue \times 2)}$$

Where *PV* is the current value of the instrument, *PV_{up}* and *PV_{down}* are the new values after the discount curve has been shocked, and *ShiftValue* is the change in interest rate. For example, if key rates of 3 months, 1, 2, 3, 5, 7, 10, 15, 20, 25, 30 years were chosen, then a 30-year bond might have corresponding key rate durations of:

3M	1Y	2Y	3Y	5Y	7Y	10Y	15Y	20Y	25Y	30Y
.01	.04	.09	.21	.4	.65	1.27	1.71	1.68	1.83	7.03

The key rate durations add up to approximately equal the duration of the bond.

Compute the key rate duration of the US Treasury Bond with maturity date of August 15, 2028 and coupon rate of 5.5%.

```
Settle = datenum('18-Nov-2008');
CouponRate = 5.500/100;
Maturity = datenum('15-Aug-2028');
Price = 114.83;
```

For the ZeroData information on the current spot curve for this bond, refer to <https://www.treasury.gov/resource-center/data-chart-center/interest-rates/Pages/TextView.aspx?data=yield>.

```
ZeroDates = daysadd(Settle , [30 90 180 360 360*2 360*3 360*5 ...  
360*7 360*10 360*20 360*30]);  
ZeroRates = ([0.06 0.12 0.81 1.08 1.22 1.53 2.32 2.92 3.68 4.42 4.20]/100)';
```

Compute the key rate duration using `bndkrdur` for a specific set of rates (choose this based on the maturities of the available hedging instruments).

```
krd = bndkrdur([ZeroDates ZeroRates], CouponRate, Settle, Maturity, 'keyrates', [2 5 10 20])
```

```
krd = 1×4
```

```
0.2865    0.8729    2.6451    8.5778
```

Note, the sum of the key rate durations approximately equals the duration of the bond.

```
[sum(krd) bnddurp(Price, CouponRate, Settle, Maturity)]
```

```
ans = 1×2
```

```
12.3823    12.3919
```

See Also

`bndconvp` | `bndconvy` | `bnddurp` | `bnddury` | `bndkrdur`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29
- “Computing Treasury Bill Price and Yield” on page 2-26

More About

- “Treasury Bills Defined” on page 2-25

Treasury Bills Defined

Treasury bills are short-term securities (issued with maturities of one year or less) sold by the United States Treasury. Sales of these securities are frequent, usually weekly. From time to time, the Treasury also offers longer duration securities called Treasury notes and Treasury bonds.

A Treasury bill is a discount security. The holder of the Treasury bill does not receive periodic interest payments. Instead, at the time of sale, a percentage discount is applied to the face value. At maturity, the holder redeems the bill for full face value.

The basis for Treasury bill interest calculation is actual/360. Under this system, interest accrues on the actual number of elapsed days between purchase and maturity, and each year contains 360 days.

See Also

`tbilldisc2yield` | `tbillprice` | `tbillrepo` | `tbillyield` | `tbillyield2disc` | `tbillval01` | `tbl2bond` | `tr2bonds` | `zbtprice` | `zbtyield`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29
- “Computing Treasury Bill Price and Yield” on page 2-26

Computing Treasury Bill Price and Yield

In this section...

“Introduction” on page 2-26
 “Treasury Bill Repurchase Agreements” on page 2-26
 “Treasury Bill Yields” on page 2-27

Introduction

Financial Toolbox software provides the following suite of functions for computing price and yield on Treasury bills.

Treasury Bill Functions

Function	Purpose
<code>tbilldisc2yield</code>	Convert discount rate to yield.
<code>tbillprice</code>	Price Treasury bill given its yield or discount rate.
<code>tbillrepo</code>	Break-even discount of repurchase agreement.
<code>tbillyield</code>	Yield and discount of Treasury bill given its price.
<code>tbillyield2disc</code>	Convert yield to discount rate.
<code>tbillval01</code>	The value of 1 basis point (one hundredth of one percentage point, or 0.0001) given the characteristics of the Treasury bill, as represented by its settlement and maturity dates. You can relate the basis point to discount, money-market, or bond-equivalent yield.

For all functions with yield in the computation, you can specify yield as money-market or bond-equivalent yield. The functions all assume a face value of \$100 for each Treasury bill.

Treasury Bill Repurchase Agreements

This example shows how to compute the break-even discount rate. This is the rate that correctly prices the Treasury bill such that the profit from selling the bill equals 0.

```
Maturity = '26-Dec-2002';
InitialDiscount = 0.0161;
PurchaseDate = '26-Sep-2002';
SaleDate = '26-Oct-2002';
RepoRate = 0.0149;

BreakevenDiscount = tbillrepo(RepoRate, InitialDiscount, ...
PurchaseDate, SaleDate, Maturity)

BreakevenDiscount =
0.0167
```

You can check the result of this computation by examining the cash flows in and out from the repurchase transaction. First compute the price of the Treasury bill on the purchase date (September 26).

```
PriceOnPurchaseDate = tbillprice(InitialDiscount, ...
PurchaseDate, Maturity, 3)
```

```
PriceOnPurchaseDate =
99.5930
```

Compute the interest due on the repurchase agreement.

```
RepoInterest = ...
RepoRate*PriceOnPurchaseDate*days360(PurchaseDate, SaleDate)/360
```

```
RepoInterest =
0.1237
```

RepoInterest for a 1.49% 30-day term repurchase agreement (30/360 basis) is **0.1237**.

Compute the price of the Treasury bill on the sale date (October 26).

```
PriceOnSaleDate = tbillprice(BreakevenDiscount, SaleDate, ...
Maturity, 3)
```

```
PriceOnSaleDate =
99.7167
```

Examining the cash flows, observe that the break-even discount causes the sum of the price on the purchase date plus the accrued 30-day interest to be equal to the price on sale date. The following table shows the cash flows:

Cash Flows from Repurchase Agreement

Date	Cash Out Flow		Cash In Flow	
9/26/2002	Purchase T-bill	99.593	Repo money	99.593
10/26/2002	Payment of repo	99.593	Sell T-bill	99.7168
	Repo interest	0.1238		
	Total	199.3098		199.3098

Treasury Bill Yields

This example shows how to convert a Treasury bill discount to an equivalent yield.

You can examine the money-market and bond-equivalent yields of a Treasury bill at the time of purchase and sale. The function `tbilldisc2yield` can perform both computations at one time.

```
Maturity = '26-Dec-2002';
InitialDiscount = 0.0161;
PurchaseDate = '26-Sep-2002';
SaleDate = '26-Oct-2002';
RepoRate = 0.0149;
BreakevenDiscount = tbillrepo(RepoRate, InitialDiscount, ...
PurchaseDate, SaleDate, Maturity)
```

```
BreakevenDiscount =
0.0167
```

```
[BEYield, MMYield] = ...  
tbilldisc2yield([InitialDiscount; BreakevenDiscount], ...  
[PurchaseDate; SaleDate], Maturity)
```

```
BEYield = 2×1
```

```
0.0164  
0.0170
```

```
MMYield = 2×1
```

```
0.0162  
0.0168
```

For the short Treasury bill (fewer than 182 days to maturity), the money-market yield is 360/365 of the bond-equivalent yield, as this example shows.

See Also

[tbilldisc2yield](#) | [tbillprice](#) | [tbillrepo](#) | [tbillyield](#) | [tbillyield2disc](#) | [tbillval01](#) | [tbl2bond](#) | [tr2bonds](#) | [zbtprice](#) | [zbtyield](#)

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Term Structure of Interest Rates” on page 2-29

More About

- “Treasury Bills Defined” on page 2-25

Term Structure of Interest Rates

This example shows how to derive and analyze interest-rate curves, including data conversion and extrapolation, bootstrapping, and interest-rate curve conversions.

One of the first problems in analyzing the term structure of interest rates is dealing with market data reported in different formats. Treasury bills, for example, are quoted with bid and asked bank-discount rates. Treasury notes and bonds, on the other hand, are quoted with bid and asked prices based on \$100 face value. To examine the full spectrum of Treasury securities, analysts must convert data to a single format. Financial Toolbox™ functions ease this conversion. The following code uses only one security each; analysts often use 30, 100, or more of each.

First, capture Treasury bill quotes and Treasury bond quotes in their reported format.

```
%      Maturity      Days Bid    Ask    AskYield
TBill = [datetime('12/26/2000') 53    0.0503  0.0499  0.0510];

%      Coupon  Maturity      Bid    Ask    AskYield
TBond = [0.08875  datetime(2001,11,5) 103+4/32  103+6/32  0.0564];
```

Note that these quotes are based on a November 3, 2000 settlement date.

```
Settle = datetime('3-Nov-2000');
```

Use the `tbl2bond` to convert the Treasury bill data to Treasury bond format.

```
TBTBond = tbl2bond(TBill)

TBTBond = 1×5
105 ×

      0      7.3085      0.0010      0.0010      0.0000
```

The second element of `TBTBond` is the serial date number for December 26, 2000.

Combine short-term (Treasury bill) with long-term (Treasury bond) data to set up the overall term structure.

```
TBondsAll = [TBTBond; TBond]

TBondsAll = 2×5
105 ×

      0      7.3085      0.0010      0.0010      0.0000
0.0000      7.3116      0.0010      0.0010      0.0000
```

Use `tr2bonds` to convert the bond data into a form ready for the bootstrapping functions. `tr2bonds` generates a matrix of bond information sorted by maturity date, plus vectors of prices and yields.

```
[Bonds, Prices, Yields] = tr2bonds(TBondsAll)

Bonds = 2×6
105 ×
```

```
7.3085      0      0.0010      0      0      0.0000
7.3116      0.0000      0.0010      0.0000      0      0.0000
```

```
Prices = 2×1
```

```
99.2654
103.1875
```

```
Yields = 2×1
```

```
0.0510
0.0564
```

Use a bootstrapping function to derive an implied zero curve. Bootstrapping is a process whereby you begin with known data points and solve for unknown data points using an underlying arbitrage theory. Every coupon bond can be valued as a package of zero-coupon bonds which mimic its cash flow and risk characteristics. By mapping yields-to-maturity for each theoretical zero-coupon bond, to the dates spanning the investment horizon, you can create a theoretical zero-rate curve. The Financial Toolbox™ software provides two bootstrapping functions: `zbtprice` derives a zero curve from bond data and *prices*, and `zbtyield` derives a zero curve from bond data and *yields*. Using `zbtprice`

```
[ZeroRates, CurveDates] = zbtprice(Bonds, Prices, Settle)
```

```
ZeroRates = 2×1
```

```
0.0516
0.0558
```

```
CurveDates = 2×1
```

```
730846
731160
```

`CurveDates` gives the investment horizon.

```
datestr(CurveDates)
```

```
ans = 2×11 char array
'26-Dec-2000'
'05-Nov-2001'
```

Use the functions `zero2disc`, `zero2fwd`, and `zero2pyld` to construct discount, forward, and par yield curves from the zero curve, and vice versa.

```
[DiscRates, CurveDates] = zero2disc(ZeroRates, CurveDates, Settle)
```

```
DiscRates = 2×1
```

```
0.9926
0.9462
```

```
CurveDates = 2×1
```

```
730846  
731160
```

```
[FwdRates, CurveDates] = zero2fwd(ZeroRates, CurveDates, Settle)
```

```
FwdRates = 2×1
```

```
0.0516  
0.0565
```

```
CurveDates = 2×1
```

```
730846  
731160
```

```
[PYldRates, CurveDates] = zero2pyld(ZeroRates, CurveDates, Settle)
```

```
PYldRates = 2×1
```

```
0.0522  
0.0557
```

```
CurveDates = 2×1
```

```
730846  
731160
```

See Also

tbilldisc2yield | tbillprice | tbillrepo | tbillyield | tbillyield2disc | tbillval01 |
tbl2bond | tr2bonds | zbtprice | zbtyield

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Computing Treasury Bill Price and Yield” on page 2-26

More About

- “Treasury Bills Defined” on page 2-25

Returns with Negative Prices

Once considered a mathematical impossibility, negative prices have become an established aspect of many financial markets. Negative prices arise in situations where investors determine that holding an asset entails more risk than the current value of the asset. For example, energy futures see negative prices because of costs associated with overproduction and limited storage capacity. In a different setting, central banks impose negative interest rates when national economies become deflationary, and the pricing of interest rate derivatives, traditionally based on positivity, have to be rethought (see “Work with Negative Interest Rates Using Functions” (Financial Instruments Toolbox)). A negative price encourages the buyer to take something from the seller, and the seller pays a fee for the service of divesting.

MathWorks® Computational Finance products support several functions for converting between price series $p(t)$ and return series $r(t)$. Price positivity is not a requirement. The returns computed from input negative prices can be unexpected, but they have mathematical meaning that can help you to understand price movements.

Negative Price Conversion

Financial Toolbox functions `ret2tick` and `tick2ret` support converting between price series $p(t)$ and return series $r(t)$.

For simple returns (default), the functions implement the formulas

$$\begin{aligned} r_s(t) &= \frac{p_s(t)}{p_s(t-1)} - 1 \\ p_s(t) &= p_s(t-1)(r_s(t) + 1). \end{aligned}$$

For continuous returns, the functions implement the formulas

$$\begin{aligned} r_c(t) &= \log\left(\frac{p_c(t)}{p_c(t-1)}\right) \\ p_c(t) &= p_c(t-1)e^{r_c(t)}. \end{aligned}$$

The functions `price2ret` and `ret2price` implement the same formulas, but they divide by Δt in the return formulas and they multiply by Δt in the price formulas. A positive factor of Δt (enforced by required monotonic observation times) does not affect the behavior of the functions. Econometrics Toolbox™ calls simple returns *periodic*, and continuous returns are the default. Otherwise, the functionality between the set of functions is identical. This example concentrates on the Financial Toolbox functions.

In the simple return formula, $r_s(t)$ is the percentage change (*PC*) in $p_s(t-1)$ over the interval $[t-1, t]$

$$\begin{aligned} PC &= \frac{p_s(t)}{p_s(t-1)} - 1 \\ p_s(t) &= p_s(t-1) + PC \cdot p_s(t-1). \end{aligned}$$

For positive prices, the range of *PC* is $(-1, \infty)$, that is, anything from a 100% loss ($p_s: p_s(t-1) \rightarrow 0$) to unlimited gain. The recursion in the second equation gives the subsequent prices; $p_s(t)$ is computed from $p_s(t-1)$ by adding a percentage of $p_s(t-1)$.

Furthermore, you can aggregate simple returns through time using the formula

$$\frac{p_s(T)}{p_s(1)} - 1 = \prod_{t=2}^T (r_s(t) + 1) - 1,$$

where the left-hand side represents the simple return over the entire interval $[0, T]$.

Continuous returns add 1 to PC to move the range to $(0, \infty)$, the domain of the real logarithm. Continuous returns have the time aggregation property

$$\log\left(\frac{p_c(T)}{p_c(1)} - 1\right) = \log\left(\prod_{t=2}^T \left(\frac{p_c(t)}{p_c(t-1)}\right)\right) = \sum_{t=2}^T \log\left(\frac{p_c(t)}{p_c(t-1)}\right) = \sum_{t=2}^T r_c(t).$$

This transformation ensures additivity of compound returns.

If negative prices are allowed, the range of simple returns PC expands to $(-\infty, \infty)$, that is, anything from unlimited loss to unlimited gain. In the formula for continuous returns, logarithms of negative numbers are unavoidable. The logarithm of a negative number is not a mathematical problem because the complex logarithm (the MATLAB default) interprets negative numbers as complex numbers with phase angle $\pm\pi$, so that, for example,

$$\begin{aligned} -2 &= 2e^{i\pi} \\ \log(-2) &= 2 + i\pi \end{aligned}$$

If $x < 0$, $\log(x) = \log(|x|) \pm i\pi$. The log of a negative number has an imaginary part of $\pm\pi$. The log of 0 is undefined because the range of the exponential $e^{i\theta}$ is positive. Therefore, zero prices (that is, free exchanges) are unsupported.

Analysis of Negative Price Returns

To illustrate negative price inputs, consider the following price series and its simple returns.

```
p = [3; 1; -2; -1; 1]
```

```
p =
```

```
3
1
-2
-1
1
```

```
rs = tick2ret(p)
```

```
rs =
```

```
-0.6667
-3.0000
-0.5000
-2.0000
```

This table summarizes the recursions.

$p_s(t-1)$	$p_s(t)$	$r_s(t)$
+3	+1	-0.6667
+1	-2	-3.0000
-2	-1	-0.5000
-1	+1	-2.0000

The returns have the correct size (66%, 300%, 50%, 200%), but do they have the correct sign? If you interpret negative returns as losses, as with the positive price series, the signs seem wrong—the last two returns should be gains (that is, if you interpret less negative to be a gain). However, if you interpret the negative returns by the formula

$$p_s(t) = p_s(t-1) + PC \cdot p_s(t-1),$$

which requires the signs, the last two negative percentage changes multiply *negative* prices $p_s(t-1)$, which produces *positive* additions to $p_s(t-1)$. Briefly, a negative return on a negative price produces a positive price movement. The returns are correct.

The round trip produced by `ret2tick` returns the original price series.

```
ps = ret2tick(rs, StartPrice=3)
```

```
ps =
```

```
3
1
-2
-1
1
```

Also, the following computations shows that time aggregation holds.

```
p(5)/p(1) - 1
```

```
ans =
```

```
-0.6667
```

```
prod(rs + 1) - 1
```

```
ans =
```

```
-0.6667
```

For continuous returns, negative price ratios $p_c(t)/p_c(t-1)$ are interpreted as complex numbers with phase angles $\pm\pi$, and the complex logarithm is invoked.

```
rc = tick2ret(p, Method="continuous")
```

```
rc =
```

```
-1.0986 + 0.0000i
0.6931 + 3.1416i
-0.6931 + 0.0000i
0.0000 - 3.1416i
```

This table summarizes the recursions.

$p_c(t-1)$	$p_c(t)$	$r_c(t)$
+3	+1	$-1.0986 + 0i$
+1	-2	$+0.6931 + \pi i$
-2	-1	$-0.6931 + 0i$
-1	+1	$0.0000 - \pi i$

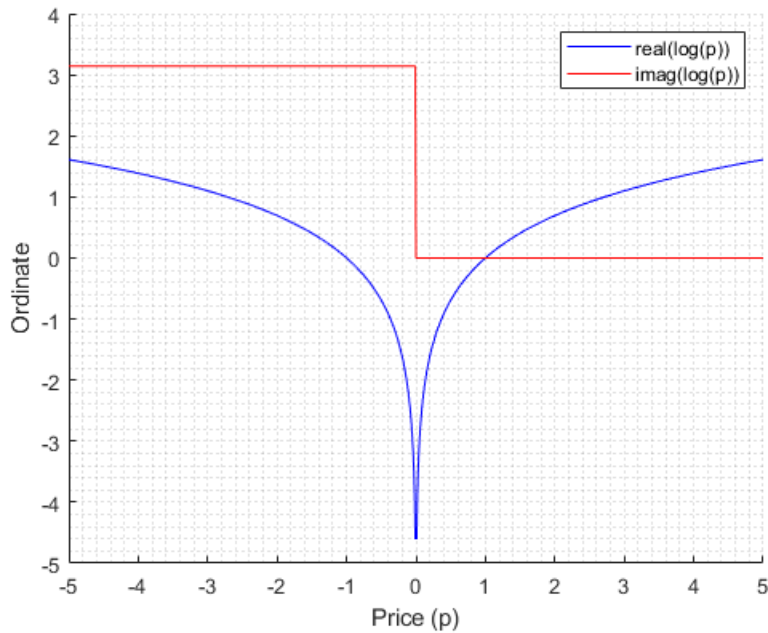
The real part shows the trend in the absolute price series. When $|p_c(t-1)| < |p_c(t)|$, that is, when prices move away from zero, $r_c(t)$ has a positive real part. When $|p_c(t-1)| > |p_c(t)|$, that is, when prices move toward zero, $r_c(t)$ has a negative real part. When $|p_c(t-1)| = |p_c(t)|$, that is, when the absolute size of the prices is unchanged, $r_c(t)$ has a zero real part. For positive price series, where the absolute series is the same as the series itself, the real part has its usual meaning.

The imaginary part shows changes of sign in the price series. When $p_c(t-1) > 0$ and $p_c(t) < 0$, that is, when prices move from investments to divestments, $r_c(t)$ has a positive imaginary part ($+\pi$). When $p_c(t-1) < 0$ and $p_c(t) > 0$, that is, when prices move from divestments to investments, $r_c(t)$ has a negative imaginary part ($-\pi$). When the sign of the prices is unchanged, $r_c(t)$ has a zero imaginary part. For positive price series, changes of sign are irrelevant, and the imaginary part conveys no information (0).

Visualization of Complex Returns

Complex continuous returns contain a lot of information. Visualizing the information can help you to interpret the complex returns. The following code plots the real and imaginary parts of the logarithm on either side of zero.

```
p = -5:0.01:5;
hold on
plot(p,real(log(p)),"b")
plot(p,imag(log(p)),"r")
xticks(-5:5)
xlabel("Price (p)")
ylabel("Ordinate")
legend(["real(log(p))" "imag(log(p))"],AutoUpdate=false)
grid minor
```



Due to the following identity

$$\begin{aligned}
 r_c(t) &= \log\left(\frac{p_c(t)}{p_c(t-1)}\right) \\
 &= \log(p_c(t)) - \log(p_c(t-1)) \\
 &= \underbrace{[\text{real}(\log(p_c(t)) - \text{real}(\log(p_c(t-1))))]}_{\text{blue curve}} + \underbrace{[\text{imag}(\log(p_c(t))) - \text{imag}(\log(p_c(t-1)))]}_{\text{red curve}} \cdot i,
 \end{aligned}$$

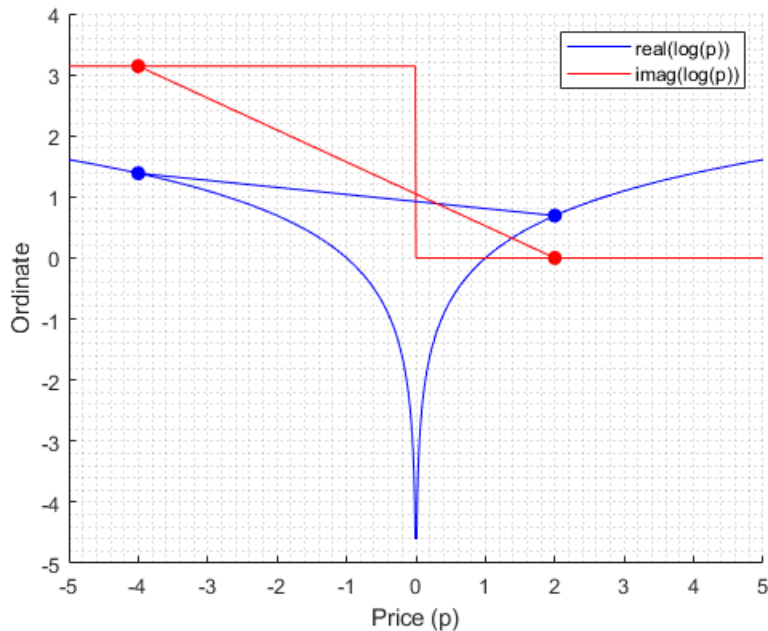
you can read the real part of a continuous return as a difference in ordinates on the blue graph, and you can read the imaginary part as a difference in ordinates on the red graph. Absolute price movements toward zero result in a negative real part and absolute price movements away from zero result in a positive real part. Likewise, changes of sign result in a jump of $\pm\pi$ in the imaginary part, with the sign change depending on the direction of the move.

For example, the plot below superimposes the real and imaginary parts of the logarithm at prices $p = -4$ and $p = 2$, with lines to help visualize their differences.

```

p = [-4; 2];
plot(p, real(log(p)), "bo-", MarkerFaceColor="b")
plot(p, imag(log(p)), "ro-", MarkerFaceColor="r")
hold off

```

If $p_c(t-1) = -4$ and $p_c(t) = 2$, the real part of $\log(p_c(t)) - \log(p_c(t-1))$ is negative (line slopes down), corresponding to a decrease in absolute price. The imaginary part is $0 - \pi = -\pi$, corresponding to a change of sign from negative to positive. If the direction of the price movement is reversed, so that $p_c(t-1) = 2$ and $p_c(t) = -4$, the positive difference in the real part corresponds to an increase in absolute price, and the positive difference in the imaginary part corresponds to a change of sign from positive to negative.

If you convert the continuous returns $r_c(t) = \log(p_c(t)/p_c(t-1))$ to simple returns $r_s = (p_s(t)/p_s(t-1) - 1)$ by the following computation, the result is the same simple returns series as before.

```
rs = exp(rc) - 1
```

```
rs =
```

```
-0.6667 + 0.0000i
-3.0000 + 0.0000i
-0.5000 + 0.0000i
-2.0000 - 0.0000i
```

You can complete the round trip, which results in the expected price series, by the computation

```
pc = ret2tick(rc, Method="continuous", StartPrice=3)
```

```
pc =
```

```
3.0000 + 0.0000i
1.0000 + 0.0000i
-2.0000 + 0.0000i
-1.0000 + 0.0000i
1.0000 + 0.0000i
```

Conclusion

Complex continuous returns are a necessary intermediary when considering logarithms of negative price ratios. `tick2ret` computes a continuous complex extension of the function on the positive real axis. The logarithm maintains the additivity property, used when computing multiperiod returns.

Because of the extensible logarithm implemented in MATLAB, current implementations of Computational Finance tools that accept prices and returns behave logically with negative prices. The interpretation of complex-valued results can be unfamiliar at first, but as shown, the results are meaningful and explicable.

See Also

`tick2ret` | `ret2tick`

More About

- “Work with Negative Interest Rates Using Functions” (Financial Instruments Toolbox)

Pricing and Analyzing Equity Derivatives

In this section...

"Introduction" on page 2-39

"Sensitivity Measures" on page 2-39

"Analysis Models" on page 2-40

Introduction

These toolbox functions compute prices, sensitivities, and profits for portfolios of options or other equity derivatives. They use the Black-Scholes model for European options and the binomial model for American options. Such measures are useful for managing portfolios and for executing collars, hedges, and straddles:

- A collar is an interest-rate option that guarantees that the rate on a floating-rate loan will not exceed a certain upper level nor fall below a lower level. It is designed to protect an investor against wide fluctuations in interest rates.
- A hedge is a securities transaction that reduces or offsets the risk on an existing investment position.
- A straddle is a strategy used in trading options or futures. It involves simultaneously purchasing put and call options with the same exercise price and expiration date, and it is most profitable when the price of the underlying security is very volatile.

Sensitivity Measures

There are six basic sensitivity measures associated with option pricing: delta, gamma, lambda, rho, theta, and vega — the "greeks." The toolbox provides functions for calculating each sensitivity and for implied volatility.

Delta

Delta of a derivative security is the rate of change of its price relative to the price of the underlying asset. It is the first derivative of the curve that relates the price of the derivative to the price of the underlying security. When delta is large, the price of the derivative is sensitive to small changes in the price of the underlying security.

Gamma

Gamma of a derivative security is the rate of change of delta relative to the price of the underlying asset; that is, the second derivative of the option price relative to the security price. When gamma is small, the change in delta is small. This sensitivity measure is important for deciding how much to adjust a hedge position.

Lambda

Lambda, also known as the elasticity of an option, represents the percentage change in the price of an option relative to a 1% change in the price of the underlying security.

Rho

Rho is the rate of change in option price relative to the risk-free interest rate.

Theta

Theta is the rate of change in the price of a derivative security relative to time. Theta is usually small or negative since the value of an option tends to drop as it approaches maturity.

Vega

Vega is the rate of change in the price of a derivative security relative to the volatility of the underlying security. When vega is large the security is sensitive to small changes in volatility. For example, options traders often must decide whether to buy an option to hedge against vega or gamma. The hedge selected usually depends upon how frequently one rebalances a hedge position and also upon the standard deviation of the price of the underlying asset (the volatility). If the standard deviation is changing rapidly, balancing against vega is preferable.

Implied Volatility

The implied volatility of an option is the standard deviation that makes an option price equal to the market price. It helps determine a market estimate for the future volatility of a stock and provides the input volatility (when needed) to the other Black-Scholes functions.

Analysis Models

Toolbox functions for analyzing equity derivatives use the Black-Scholes model for European options and the binomial model for American options. The Black-Scholes model makes several assumptions about the underlying securities and their behavior. The Black-Scholes model was the first complete mathematical model for pricing options, developed by Fischer Black and Myron Scholes. It examines market price, strike price, volatility, time to expiration, and interest rates. It is limited to only certain kinds of options.

The binomial model, on the other hand, makes far fewer assumptions about the processes underlying an option. A binomial model is a method of pricing options or other equity derivatives in which the probability over time of each possible price follows a binomial distribution. The basic assumption is that prices can move to only two values (one higher and one lower) over any short time period. For further explanation, see *Options, Futures, and Other Derivatives* by John Hull in “Bibliography” on page A-2.

Black-Scholes Model

This example shows how to compute the call and put prices of a European option and its delta, gamma, lambda, and implied volatility.

Using the Black-Scholes model entails several assumptions:

- The prices of the underlying asset follow an Ito process. (See “Derivatives Pricing and Yields” on page A-3, page 222.)
- The option can be exercised only on its expiration date (European option).
- Short selling is permitted.
- There are no transaction costs.
- All securities are divisible.
- There is no riskless arbitrage (where *arbitrage* is the purchase of securities on one market for immediate resale on another market to profit from a price or currency discrepancy).

- Trading is a continuous process.
- The risk-free interest rate is constant and remains the same for all maturities.

If any of these assumptions is untrue, Black-Scholes may not be an appropriate model.

To illustrate toolbox Black-Scholes functions (`blsprice`, `blsdelta`, `blsgamma`, `blsvega`, and `blslambda`) this example computes the call and put prices of a European option and its delta, gamma, lambda, and implied volatility. The asset price is \$100.00, the exercise price is \$95.00, the risk-free interest rate is 10%, the time to maturity is 0.25 years, the volatility is 0.50, and the dividend rate is 0.

```
[OptCall, OptPut] = blsprice(100, 95, 0.10, 0.25, 0.50, 0)
```

```
OptCall =  
13.6953
```

```
OptPut =  
6.3497
```

```
[CallVal, PutVal] = blsdelta(100, 95, 0.10, 0.25, 0.50, 0)
```

```
CallVal =  
0.6665
```

```
PutVal =  
-0.3335
```

```
GammaVal = blsgamma(100, 95, 0.10, 0.25, 0.50, 0)
```

```
GammaVal =  
0.0145
```

```
VegaVal = blsvega(100, 95, 0.10, 0.25, 0.50, 0)
```

```
VegaVal =  
18.1843
```

```
[LamCall, LamPut] = blslambda(100, 95, 0.10, 0.25, 0.50, 0)
```

```
LamCall =  
4.8664
```

```
LamPut =  
-5.2528
```

To summarize:

- The option call price `OptCall` = \$13.70
- The option put price `OptPut` = \$6.35
- delta for a call `CallVal` = 0.6665 and delta for a put `PutVal` = -0.3335
- gamma `GammaVal` = 0.0145
- vega `VegaVal` = 18.1843
- lambda for a call `LamCall` = 4.8664 and lambda for a put `LamPut` = -5.2528

As a computation check, find the implied volatility of the option using the call option price from `blsprice`.

```
Volatility = blsimpv(100, 95, 0.10, 0.25, OptCall)
```

```
Volatility =  
0.5000
```

The function returns an implied volatility of 0.500, the original `blsprice` input.

Binomial Model

This example shows how to price an American call option using a binomial model.

The binomial model for pricing options or other equity derivatives assumes that the probability over time of each possible price follows a binomial distribution. The basic assumption is that prices can move to only two values, one up and one down, over any short time period. Plotting the two values, and then the subsequent two values each, and then the subsequent two values each, and so on over time, is known as "building a binomial tree.". This model applies to American options, which can be exercised any time up to and including their expiration date.

This example prices an American call option using a binomial model. The asset price is \$100.00, the exercise price is \$95.00, the risk-free interest rate is 10%, and the time to maturity is 0.25 years. The function `binprice` computes the tree in increments of 0.05 years, so there are $0.25/0.05 = 5$ periods in the example. The volatility is 0.50, this is a call (`flag = 1`), the dividend rate is 0, and it pays a dividend of \$5.00 after three periods (an ex-dividend date).

```
[StockPrice, OptionPrice] = binprice(100, 95, 0.10, 0.25, ...  
0.05, 0.50, 1, 0, 5.0, 3)
```

```
StockPrice = 6×6
```

100.0000	111.2713	123.8732	137.9629	148.6915	166.2807
0	89.9677	100.0495	111.3211	118.8981	132.9629
0	0	80.9994	90.0175	95.0744	106.3211
0	0	0	72.9825	76.0243	85.0175
0	0	0	0	60.7913	67.9825
0	0	0	0	0	54.3608

```
OptionPrice = 6×6
```

12.1011	19.1708	29.3470	42.9629	54.1653	71.2807
0	5.3068	9.4081	16.3211	24.3719	37.9629
0	0	1.3481	2.7402	5.5698	11.3211
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

The output from the binomial function is a binary tree. Read the `StockPrice` matrix this way: column 1 shows the price for period 0, column 2 shows the up and down prices for period 1, column 3 shows the up-up, up-down, and down-down prices for period 2, and so on. Ignore the zeros. The `OptionPrice` matrix gives the associated option value for each node in the price tree. Ignore the zeros that correspond to a zero in the price tree.

See Also

`blsprice` | `binprice` | `blkimpv` | `blkprice` | `blsdelta` | `blsgamma` | `blsimpv` | `blslambda` | `blsrho` | `blstheta` | `blsvega` | `opprofit`

Related Examples

- “Handle and Convert Dates” on page 2-2
- “Greek-Neutral Portfolios of European Stock Options” on page 10-14
- “Plotting Sensitivities of an Option” on page 10-25
- “Plotting Sensitivities of a Portfolio of Options” on page 10-27

About Life Tables

Life tables are used for life insurance and work with the probability distribution of human mortality. This distribution, which is age-dependent, has several characteristic features that are consequences of biological, cultural, and behavioral factors. Usually, the practitioners of life studies use life tables that contain age-dependent series for specific demographics. The tables are in a standard format with standard notation that is specific to the life studies field. An example of a life table is shown in Table 1 from CDC life tables for the United States.

Table 1. Life table for the total population: United States, 2009

Age (years)	Probability of dying between ages x and $x + 1$	Number surviving to age x	Number dying between ages x and $x + 1$	Person-years lived between ages x and $x + 1$	Total number of person-years lived above age x	Expectation of life at age x
	q_x	l_x	d_x	L_x	T_x	e_x
0-1	0.006372	100,000	637	99,444	7,946,926	78.5
1-2	0.000407	99,363	40	99,343	7,747,481	78.0
2-3	0.000274	99,322	27	99,309	7,548,139	77.0
3-4	0.000209	99,295	21	99,285	7,548,830	76.0
4-5	0.000160	99,274	16	99,266	7,449,545	75.0
5-6	0.000150	99,259	15	99,251	7,350,279	74.1
6-7	0.000135	99,244	13	99,237	7,251,028	73.1
7-8	0.000122	99,230	12	99,224	7,151,791	72.1
8-9	0.000109	99,215	11	99,213	7,052,566	71.1
9-10	0.000095	99,207	9	99,203	6,953,354	70.1
10-11	0.000087	99,198	9	99,194	6,854,151	69.1
11-12	0.000093	99,189	9	99,185	6,754,957	68.1
12-13	0.000127	99,180	13	99,174	6,655,773	67.1
13-14	0.000160	99,167	16	99,158	6,556,599	66.1
14-15	0.000279	99,148	28	99,134	6,457,441	65.1
15-16	0.000370	99,121	37	99,102	6,358,307	64.1
16-17	0.000454	99,084	45	99,061	6,259,205	63.2
17-18	0.000537	99,039	53	99,012	6,160,143	62.2
18-19	0.000615	98,986	61	98,955	6,061,131	61.2
19-20	0.000691	98,925	68	98,891	5,962,175	60.3

Often, these life tables can have numerous variations such as abridged tables (which pose challenges due to the granularity of the data) and different termination criteria (that can make it difficult to compare tables or to compute life expectancies).

Most raw life tables have one or more of the first three series in this table (q_x , l_x , and d_x) and the notation for these three series is standard in the field.

- The q_x series is basically the discrete hazard function for human mortality.
- The l_x series is the survival function multiplied by a radix of 100,000.
- The d_x series is the discrete probability density for the distribution as a function of age.

Financial Toolbox can handle arbitrary life table data supporting several standard models of mortality and provides various interpolation methods to calibrate and analyze the life table data.

Although primarily designed for life insurance applications, the life tables functions (`lifetableconv`, `lifetablefit`, and `lifetablegen`) can also be used by social scientists, behavioral psychologists, public health officials, and medical researchers.

Life Tables Theory

Life tables are based on hazard functions and survival functions which are, in turn, derived from probability distributions. Specifically, given a continuous probability distribution, its cumulative distribution function is $F(x)$ and its probability density function is $f(x) = dF(x)/dx$.

For the analysis of mortality, the random variable of interest X is the distribution of ages at which individuals die within a population. So, the probability that someone dies by age x is

$$\Pr[X \leq x] = F(x)$$

The survival function, $(s(x))$, which characterizes the probability that an individual lives beyond a specified age $x > 0$, is

$$\begin{aligned}s(x) &= \Pr[X > x] \\ &= 1 - F(x)\end{aligned}$$

For a continuous probability distribution, the hazard function is a function of the survival function with

$$\begin{aligned}h(x) &= \lim_{\Delta x \rightarrow 0} \frac{\Pr[x \leq X < x + \Delta x | X \geq x]}{\Delta x} \\ &= -\frac{1}{s(x)} \frac{d(s(x))}{dx}\end{aligned}$$

and the survival functions is a function of the hazard function with

$$s(x) = \exp\left(-\int_0^x h(\xi) d\xi\right)$$

Life table models generally specify either the hazard function or the survival function. However, life tables are discrete and work with discrete versions of the hazard and survival functions. Three series are used for life tables and the notation is the convention. The discrete hazard function is denoted as

$$\begin{aligned}q_x &\approx h(x) \\ &= 1 - \frac{s(x+1)}{s(x)}\end{aligned}$$

which is the probability a person at age x dies by age $x + 1$ (where x is in years). The discrete survival function is presented in terms of an initial number of survivors at birth called the life table radix (which is usually 100,000 individuals) and is denoted as

$$l_x = l_0 s(x)$$

with radix $l_0 = 100000$. This number, l_x , represents the number of individuals out of 100,000 at birth who are still alive at age x .

A third series is related to the probability density function which is the number of "standardized" deaths in a given year denoted as

$$d_x = l_x - l_{x+1}$$

Based on a few additional rules about how to initialize and terminate these series, any one series can be derived from any of the other series.

See Also

[lifetableconv](#) | [lifetablefit](#) | [lifetablegen](#)

Related Examples

- "Case Study for Life Tables Analysis" on page 2-46

Case Study for Life Tables Analysis

This example shows how to use the basic workflow for life tables.

Load the life table data file.

```
load us_lifetable_2009
```

Calibrate life table from survival data with the default heligman-pollard parametric model.

```
a = lifetablefit(x, lx);
```

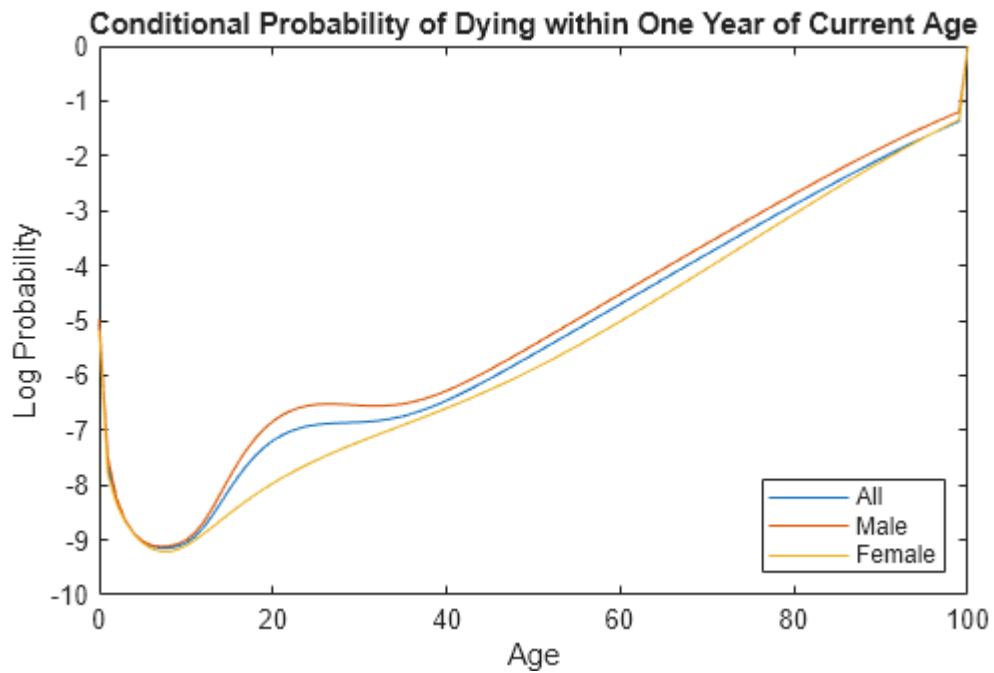
Generate life table series from the calibrated mortality model.

```
qx = lifetablegen((0:100), a);  
display(qx(1:40,:))
```

0.0063	0.0069	0.0057
0.0005	0.0006	0.0004
0.0002	0.0003	0.0002
0.0002	0.0002	0.0002
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0001	0.0001	0.0001
0.0002	0.0002	0.0001
0.0002	0.0002	0.0002
0.0002	0.0003	0.0002
0.0003	0.0004	0.0002
0.0004	0.0005	0.0002
0.0005	0.0006	0.0003
0.0006	0.0008	0.0003
0.0007	0.0009	0.0003
0.0008	0.0011	0.0003
0.0008	0.0012	0.0004
0.0009	0.0013	0.0004
0.0009	0.0014	0.0005
0.0010	0.0014	0.0005
0.0010	0.0015	0.0005
0.0010	0.0015	0.0006
0.0010	0.0015	0.0006
0.0010	0.0015	0.0007
0.0010	0.0014	0.0007
0.0011	0.0014	0.0007
0.0011	0.0014	0.0008
0.0011	0.0014	0.0008
0.0011	0.0014	0.0009
0.0011	0.0014	0.0009
0.0012	0.0015	0.0010
0.0012	0.0015	0.0011
0.0013	0.0016	0.0011
0.0014	0.0017	0.0012
0.0015	0.0018	0.0013

Plot the qx series and display the legend. The series qx is the conditional probability that a person at age x will die between age x and the next age in the series

```
plot((0:100), log(qx));
legend(series, 'location', 'southeast');
title('Conditional Probability of Dying within One Year of Current Age');
xlabel('Age');
ylabel('Log Probability');
```



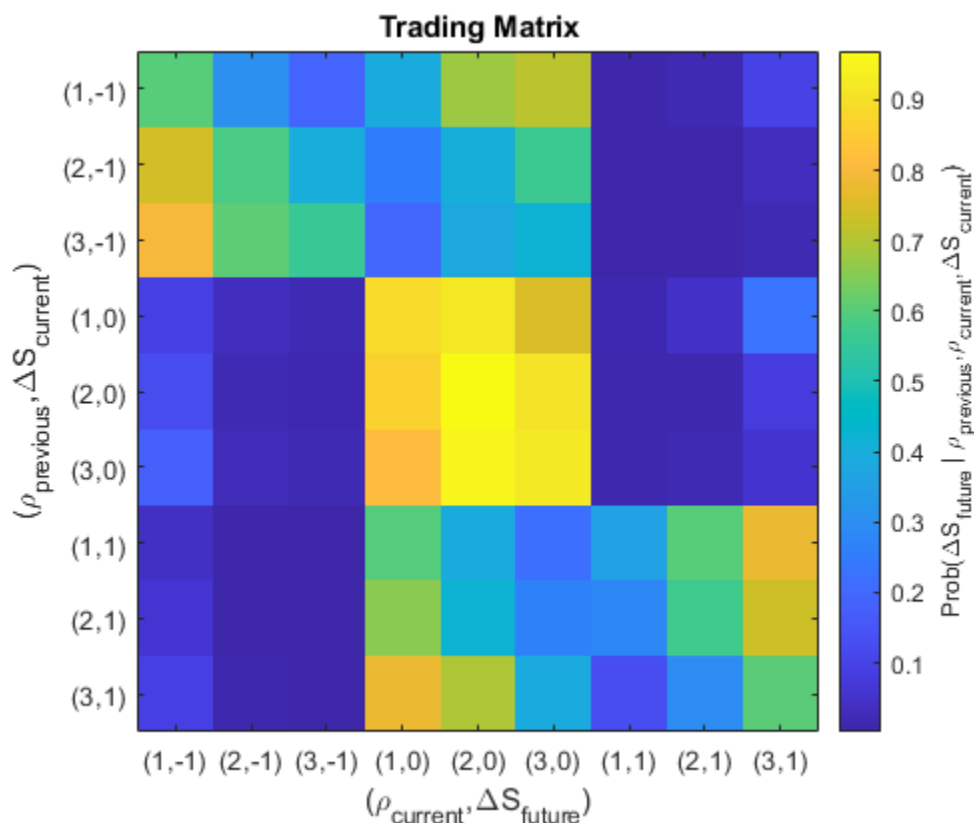
See Also

[lifetableconv](#) | [lifetablefit](#) | [lifetablegen](#)

More About

- "About Life Tables" on page 2-44

Machine Learning for Statistical Arbitrage: Introduction



Machine learning techniques for processing large amounts of data are broadly applicable in computational finance. The series of examples introduced in this topic provides a general workflow, illustrating how capabilities in MATLAB apply to a specific problem in financial engineering. The workflow is problem-oriented, exploratory, and guided by the data and the resulting analysis. The overall approach, however, is useful for constructing applications in many areas.

The workflow consists of these actions:

- Formulate a simple approach to algorithmic trading, through an analysis of market microstructure, with the goal of identifying real-time arbitrage opportunities.
- Use a large sample of exchange data to track order dynamics of a single security on a single day, selectively processing the data to develop relevant statistical measures.
- Create a model of intraday dynamics conditioned on a selection of hyperparameters introduced during feature engineering and development.
- Evaluate hyperparameter tunings using a supervising objective that computes cash returned on a model-based trading strategy.
- Optimize the trading strategy using different machine learning algorithms.
- Suggest modifications for further development.

The workflow is separated into three examples:

- 1** “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- 2** “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59
- 3** “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69

For more information about general workflows for machine learning, see:

- “Machine Learning in MATLAB”
- “Supervised Learning Workflow and Algorithms”

Machine Learning for Statistical Arbitrage I: Data Management and Visualization

This example shows techniques for managing, processing, and visualizing large amounts of financial data in MATLAB®. It is part of a series of related examples on machine learning for statistical arbitrage (see “Machine Learning Applications”).

Working with Big Data

Financial markets, with electronic exchanges such as NASDAQ executing orders on a timescale of milliseconds, generate vast amounts of data. Data streams can be mined for statistical arbitrage opportunities, but traditional methods for processing and storing dynamic analytic information can be overwhelmed by big data. Fortunately, new computational approaches have emerged, and MATLAB has an array of tools for implementing them.

Main computer memory provides high-speed access but limited capacity, whereas external storage offers low-speed access but potentially unlimited capacity. Computation takes place in memory. The computer recalls data and results from external storage.

Data Files

This example uses one trading day of NASDAQ exchange data [2] on one security (INTC) in a sample provided by LOBSTER [1] and included with Financial Toolbox™ documentation in the zip file `LOBSTER_SampleFile_INTC_2012-06-21_5.zip`. Extract the contents of the zip file into your current folder. The expanded files, including two CSV files of data and the text file `LOBSTER_SampleFiles_ReadMe.txt`, consume 93.7 MB of memory.

```
unzip("LOBSTER_SampleFile_INTC_2012-06-21_5.zip");
```

The data describes the intraday evolution of the *limit order book* (LOB), which is the record of *market orders* (best price), *limit orders* (designated price), and resulting buys and sells. The data includes the precise time of these events, with orders tracked from arrival until cancellation or execution. At each moment in the trading day, orders on both the buy and sell side of the LOB exist at various *levels* away from the midprice between the lowest ask (order to sell) and the highest bid (order to buy).

Level 5 data (five levels away from the midprice on either side) is contained in two CSV files. Extract the trading date from the message file name.

```
MSGFileName = "INTC_2012-06-21_34200000_57600000_message_5.csv"; % Message file (description of  
LOBFileName = "INTC_2012-06-21_34200000_57600000_orderbook_5.csv"; % Data file  
  
[ticker,rem] = strtok(MSGFileName,'_');  
date = strtok(rem,'_');
```

Data Storage

Daily data streams accumulate and need to be stored. A *datastore* is a repository for collections of data that are too big to fit in memory.

Use `tabularTextDatastore` to create datastores for the message and data files. Because the files contain data with different formats, create the datastores separately. Ignore generic column headers (for example, `VarName1`) by setting the `'ReadVariableNames'` name-value argument to `false`. Replace the headers with descriptive variable names obtained from

LOBSTER_SampleFiles_ReadMe.txt. Set the 'ReadSize' name-value argument to 'file' to allow similarly formatted files to be appended to existing datastores at the end of each trading day.

```
DSMSG = tabularTextDatastore(MSGFileName, 'ReadVariableNames', false, 'ReadSize', 'file');
DSMSG.VariableNames = ["Time", "Type", "OrderID", "Size", "Price", "Direction"];
```

```
DSL0B = tabularTextDatastore(LOBFileName, 'ReadVariableNames', false, 'ReadSize', 'file');
DSL0B.VariableNames = ["AskPrice1", "AskSize1", "BidPrice1", "BidSize1", ...
    "AskPrice2", "AskSize2", "BidPrice2", "BidSize2", ...
    "AskPrice3", "AskSize3", "BidPrice3", "BidSize3", ...
    "AskPrice4", "AskSize4", "BidPrice4", "BidSize4", ...
    "AskPrice5", "AskSize5", "BidPrice5", "BidSize5"];
```

Create a combined datastore by selecting Time and the level 3 data.

```
TimeVariable = "Time";
DSMSG.SelectedVariableNames = TimeVariable;

LOB3Variables = ["AskPrice1", "AskSize1", "BidPrice1", "BidSize1", ...
    "AskPrice2", "AskSize2", "BidPrice2", "BidSize2", ...
    "AskPrice3", "AskSize3", "BidPrice3", "BidSize3"];
DSL0B.SelectedVariableNames = LOB3Variables;

DS = combine(DSMSG, DSL0B);
```

You can preview the first few rows in the combined datastore without loading data into memory.

```
DSPreview = preview(DS);
LOBPreview = DSPreview(:, 1:5)
```

LOBPreview=8x5 table

Time	AskPrice1	AskSize1	BidPrice1	BidSize1
34200	2.752e+05	66	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300

The preview shows asks and bids *at the touch*, meaning the level 1 data, which is closest to the midprice. Time units are seconds after midnight, price units are dollar amounts times 10,000, and size units are the number of shares (see LOBSTER_SampleFiles_ReadMe.txt).

Tall Arrays and Timetables

Tall arrays work with out-of-memory data backed by a datastore using the MapReduce technique (see “Tall Arrays for Out-of-Memory Data”). When you use MapReduce, tall arrays remain unevaluated until you execute specific computations that use the data.

Set the execution environment for MapReduce to the local MATLAB session, instead of using Parallel Computing Toolbox™, by calling `mapreducer(0)`. Then, create a tall array from the datastore DS by using `tall`. Preview the data in the tall array.

```
mapreducer(0)
DT = tall(DS);
```

```
DTPreview = DT(:,1:5)
```

```
DTPreview =
```

M×5 tall table

Time	AskPrice1	AskSize1	BidPrice1	BidSize1
34200	2.752e+05	66	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	400
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
34200	2.752e+05	166	2.751e+05	300
:	:	:	:	:
:	:	:	:	:

Timetables allow you to perform operations specific to time series (see “Create Timetables”). Because the LOB data consists of concurrent time series, convert DT to a tall timetable.

```
DT.Time = seconds(DT.Time); % Cast time as a duration from midnight.
DTT = table2timetable(DT);
```

```
DTTPreview = DTT(:,1:4)
```

```
DTTPreview =
```

M×4 tall timetable

Time	AskPrice1	AskSize1	BidPrice1	BidSize1
34200 sec	2.752e+05	66	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	400
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
34200 sec	2.752e+05	166	2.751e+05	300
:	:	:	:	:
:	:	:	:	:

Display all variables in the MATLAB workspace.

```
whos
```

Name	Size	Bytes	Class	Attributes
DS	1x1	8	matlab.io.datastore.CombinedDatastore	
DSL0B	1x1	8	matlab.io.datastore.TabularTextDatastore	
DSMSG	1x1	8	matlab.io.datastore.TabularTextDatastore	
DSPreview	8x13	4899	table	

DT	Mx13	5292	tall
DTPreview	Mx5	2926	tall
DTT	Mx12	5056	tall
DTTPreview	Mx4	2704	tall
LOB3Variables	1x12	952	string
LOBFileName	1x1	262	string
LOBPreview	8x5	2331	table
MSGFileName	1x1	246	string
TimeVariable	1x1	166	string
date	1x1	182	string
rem	1x1	246	string
ticker	1x1	166	string

Because all the data is in the datastore, the workspace uses little memory.

Preprocess and Evaluate Data

Tall arrays allow preprocessing, or *queuing*, of computations before they are evaluated, which improves memory management in the workspace.

Midprice S and imbalance index I are used to model LOB dynamics. To queue their computations, define them, and the time base, in terms of DTT.

```
timeBase = DTT.Time;
MidPrice = (DTT.BidPrice1 + DTT.AskPrice1)/2;

% LOB level 3 imbalance index:

lambda = 0.5; % Hyperparameter
weights = exp(-(lambda)*[0 1 2]);
VAsk = weights(1)*DTT.AskSize1 + weights(2)*DTT.AskSize2 + weights(3)*DTT.AskSize3;
VBid = weights(1)*DTT.BidSize1 + weights(2)*DTT.BidSize2 + weights(3)*DTT.BidSize3;
ImbalanceIndex = (VBid-VAsk)./(VBid+VAsk);
```

The imbalance index is a weighted average of ask and bid volumes on either side of the midprice [3]. The imbalance index is a potential indicator of future price movements. The variable `lambda` is a *hyperparameter*, which is a parameter specified before training rather than estimated by the machine learning algorithm. A hyperparameter can influence the performance of the model. *Feature engineering* is the process of choosing domain-specific hyperparameters to use in machine learning algorithms. You can tune hyperparameters to optimize a trading strategy.

To bring preprocessed expressions into memory and evaluate them, use the `gather` function. This process is called *deferred evaluation*.

```
[t,S,I] = gather(timeBase,MidPrice,ImbalanceIndex);
```

```
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 4.5 sec
Evaluation completed in 5 sec
```

A single call to `gather` evaluates multiple preprocessed expressions with a single pass through the datastore.

Determine the sample size, which is the number of *ticks*, or updates, in the data.

```
numTicks = length(t)
```

```
numTicks =
581030
```

The daily LOB data contains 581,030 ticks.

Checkpoint Data

You can save both unevaluated and evaluated data to external storage for later use.

Prepend the time base with the date, and cast the result as a datetime array. Save the resulting datetime array, `MidPrice`, and `ImbalanceIndex` to a MAT-file in a specified location.

```
dateTimeBase = datetime(date) + timeBase;
Today = timetable(dateTimeBase, MidPrice, ImbalanceIndex)
```

Today =

581,030×2 tall timetable

dateTimeBase	MidPrice	ImbalanceIndex
21-Jun-2012 09:30:00	2.7515e+05	-0.205
21-Jun-2012 09:30:00	2.7515e+05	-0.26006
21-Jun-2012 09:30:00	2.7515e+05	-0.26006
21-Jun-2012 09:30:00	2.7515e+05	-0.086772
21-Jun-2012 09:30:00	2.7515e+05	-0.15581
21-Jun-2012 09:30:00	2.7515e+05	-0.35382
21-Jun-2012 09:30:00	2.7515e+05	-0.19084
21-Jun-2012 09:30:00	2.7515e+05	-0.19084
:	:	:
:	:	:

```
location = fullfile(pwd, "ExchangeData", ticker, date);
write(location, Today, 'FileType', 'mat')
```

```
Writing tall data to folder C:\TEMP\tp5b508469\finance-ex97702880\ExchangeData\INTC\2012-06-21
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 5.8 sec
Evaluation completed in 6.7 sec
```

The file is written once, at the end of each trading day. The code saves the data to a file in a date-stamped folder. The series of `ExchangeData` subfolders serves as a historical data repository.

Alternatively, you can save workspace variables evaluated with `gather` directly to a MAT-file in the current folder.

```
save("LOBVars.mat", "t", "S", "I")
```

In preparation for model validation later on, evaluate and add market order prices to the same file.

```
[MOBid, MOAsk] = gather(DTT.BidPrice1, DTT.AskPrice1);
```

```
Evaluating tall expression using the Local MATLAB Session:
- Pass 1 of 1: Completed in 4.1 sec
Evaluation completed in 4.2 sec
```

```
save("LOBVars.mat", "MOBid", "MOAsk", "-append")
```

The remainder of this example uses only the unevaluated tall timetable DTT. Clear other variables from the workspace.

```
clearvars -except DTT
whos
```

Name	Size	Bytes	Class	Attributes
DTT	581,030x12	5056	tall	

Data Visualization

To visualize large amounts of data, you must summarize, bin, or sample the data in some way to reduce the number of points plotted on the screen.

LOB Snapshot

One method of visualization is to evaluate only a selected subsample of the data. Create a snapshot of the LOB at a specific time of day (11 AM).

```
sampleTimeTarget = seconds(11*60*60); % Seconds after midnight
sampleTimes = withtol(sampleTimeTarget,seconds(1)); % 1 second tolerance
sampleLOB = DTT(sampleTimes,:);
```

```
numTimes = gather(size(sampleLOB,1))
```

Evaluating tall expression using the Local MATLAB Session:

- Pass 1 of 1: Completed in 4.1 sec

Evaluation completed in 4.4 sec

```
numTimes =
23
```

There are 23 ticks within one second of 11 AM. For the snapshot, use the tick closest to the midtime.

```
sampleLOB = sampleLOB(round(numTimes/2),:);
sampleTime = sampleLOB.Time;
```

```
sampleBidPrices = [sampleLOB.BidPrice1,sampleLOB.BidPrice2,sampleLOB.BidPrice3];
sampleBidSizes = [sampleLOB.BidSize1,sampleLOB.BidSize2,sampleLOB.BidSize3];
sampleAskPrices = [sampleLOB.AskPrice1,sampleLOB.AskPrice2,sampleLOB.AskPrice3];
sampleAskSizes = [sampleLOB.AskSize1,sampleLOB.AskSize2,sampleLOB.AskSize3];
```

```
[sampleTime,sampleBidPrices,sampleBidSizes,sampleAskPrices,sampleAskSizes] = ...
gather(sampleTime,sampleBidPrices,sampleBidSizes,sampleAskPrices,sampleAskSizes);
```

Evaluating tall expression using the Local MATLAB Session:

- Pass 1 of 2: Completed in 3.7 sec

- Pass 2 of 2: Completed in 4 sec

Evaluation completed in 8.6 sec

Visualize the limited data sample returned by gather by using bar.

```
figure
hold on

bar((sampleBidPrices/10000),sampleBidSizes,'r')
bar((sampleAskPrices/10000),sampleAskSizes,'g')
hold off
```

```

xlabel("Price (Dollars)")
ylabel("Number of Shares")
legend(["Bid", "Ask"], 'Location', 'North')
title(strcat("Level 3 Limit Order Book: ", datestr(sampleTime, "HH:MM:SS")))

```



Depth of Market

Some visualization functions work directly with tall arrays and do not require the use of `gather` (see “Visualization of Tall Arrays”). The functions automatically sample data to decrease pixel density. Visualize the level 3 intraday *depth of market*, which shows the time evolution of liquidity, by using `plot` with the tall timetable `DTT`.

```

figure
hold on

plot(DTT.Time, -DTT.BidSize1, 'Color', [1.0 0 0], 'LineWidth', 2)
plot(DTT.Time, -DTT.BidSize2, 'Color', [0.8 0 0], 'LineWidth', 2)
plot(DTT.Time, -DTT.BidSize3, 'Color', [0.6 0 0], 'LineWidth', 2)

plot(DTT.Time, DTT.AskSize1, 'Color', [0 1.0 0], 'LineWidth', 2)
plot(DTT.Time, DTT.AskSize2, 'Color', [0 0.8 0], 'LineWidth', 2)
plot(DTT.Time, DTT.AskSize3, 'Color', [0 0.6 0], 'LineWidth', 2)

hold off

xlabel("Time")
ylabel("Number of Shares")
title("Depth of Market: Intraday Evolution")
legend(["Bid1", "Bid2", "Bid3", "Ask1", "Ask2", "Ask3"], 'Location', 'NorthOutside', 'Orientation', 'Horizontal')

```